

Dragoș-Daniel DEGERAT

PROIECT DE DIPLOMĂ

**Multimodal Search: Image Retrieval Based on Text
Descriptions**

SPECIALIZAREA TEHNOLOGII ȘI SISTEME DE TELECOMUNICAȚII

*Marchează dacă ai studiat
în limba engleză:*

ÎN LIMBA ENGLEZĂ



FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII
ȘI TEHNOLOGIA INFORMAȚIEI
UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI
Specializarea: Tehnologii și Sisteme de Telecomunicații (în limba engleză)

**Multimodal Search: Image Retrieval Based on Text
Descriptions**

Proiect de diplomă

Absolvent,
Dragoș-Daniel DEGERAT



Președinte comisie,
Prof.dr.ing. Virgil DOBROTĂ

Decan,
Prof.dr.ing. Ovidiu POP

2026

Această pagină a fost lăsată goală în mod intenționat.

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI

Departamentul Comunicații

Titlul proiectului de diplomă:

Multimodal Search: Image Retrieval Based on Text Descriptions

Descrierea temei:

The objective of this work is to develop a multimodal search system that retrieves, from an image collection, the images that best match a free-form textual description provided by the user. The system is based on the pre-trained CLIP ViT-B/32 dual encoder, which projects text and images into a shared embedding space, where candidates are ranked by cosine similarity. An additional stage re-ranks the retrieved images based on their captions in the text embedding space, refining the initial results. The system is implemented in Python using PyTorch and the Hugging Face Transformers library and is evaluated on the Flickr30k dataset, containing 31,783 images, without additional model training. Performance is evaluated both qualitatively and quantitatively using similarity scores and Recall@K, while also analyzing the effect of collection size on retrieval results. The final outcome is a functional system demonstrated through an interactive graphical interface in which users can enter text queries and visualize the retrieved images.

Locul de realizare:

Locuința personală

Adaugă
consultant: ☐

Data emiterii temei: 29.10.2025

Data predării temei: 08.07.2026

Absolvent,

Dragoș-Daniel DEGERAT



Conducător,

Director departament,

Prof.dr.ing. Virgil DOBROTĂ

Sl.dr.ing Ștefania-Ramona BENEĂ



Această pagină a fost lăsată goală în mod intenționat.

Declarație pe proprie răspundere privind autenticitatea lucrării de diplomă

Subsemnata/ul **Dragoș-Daniel DEGERAT**

legitimată cu CI seria ZS nr. 194940 (conform copiei anexate prezentei declarații,
copie semnată și certificată "conform cu originalul"), autorul lucrării

Multimodal Search: Image Retrieval Based on Text Descriptions

elaborată în vederea susținerii examenului de finalizare a studiilor de licență,
la Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Programul de studiu Tehnologii și Sisteme de Telecomunicații (în limba engleză)
din cadrul Universității Tehnice din Cluj-Napoca
sesiunea iulie a anului universitar 2025-2026

declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza
cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în
bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost
folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.
Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen
de absolvire/licență/diplomă/disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative,
respectiv anularea examenului de diplomă.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea
autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și conform
copiei documentului de identitate menționat mai sus.

Data

08.07.2026

Nume și prenume

Dragoș-Daniel DEGERAT

Semnătura



Această pagină a fost lăsată goală în mod intenționat.

Absolvent: Dragoș-Daniel DEGERAT

Conducător: Sl.dr.ing Ștefania-Ramona BENEĂ

SINTEZA PROIECTULUI DE DIPLOMĂ

Multimodal Search: Image Retrieval Based on Text Descriptions

This paper addresses multimodal search: retrieving images based on free-form textual descriptions. The topic is both current and useful, as recent vision-language models such as CLIP have made it possible to search image collections without relying on manually assigned tags. The author's contribution is a complete working retrieval system that requires no additional model training. The system uses the Flickr30k dataset, containing 31,783 images, and the pre-trained CLIP ViT-B/32 dual encoder to represent images and text queries in a shared embedding space. Images are ranked according to their cosine similarity to the query. The baseline retrieval pipeline is extended with a caption-based re-ranking stage designed to improve cases where the overall scene matches the query but the requested subject is missing. The system also includes an evaluation methodology combining qualitative analysis with quantitative Recall@K measurements and an interactive graphical interface built with Gradio. The complete pipeline runs as inference on a consumer laptop. The experiments show that the average top-5 similarity score increases from 0.2647 for 500 images to 0.3196 for the full collection. In the quantitative evaluation, using dataset captions as queries, Recall@1 reaches 59.16% on a subset of 1,000 images.

Căutarea multimodală: regăsirea imaginilor pe baza descrierilor textuale

Această lucrare abordează căutarea multimodală, mai exact regăsirea imaginilor pe baza unor descrieri textuale formulate liber. Tema este actuală și utilă deoarece modelele recente de tip vision-language, precum CLIP, au făcut posibilă căutarea în colecții de imagini fără a depinde de etichete manuale. Contribuția autorului constă în realizarea unui sistem complet și funcțional de regăsire a imaginilor, fără antrenarea suplimentară a unui model. Sistemul utilizează setul de date Flickr30k, care conține 31.783 de imagini, și modelul pre-antrenat CLIP ViT-B/32 pentru a reprezenta imaginile și interogările textuale într-un spațiu comun de reprezentare. Imaginile sunt ordonate în funcție de similaritatea cosinus față de interogare. Pipeline-ul de bază este extins cu o etapă de reordonare bazată pe descrierile imaginilor, concepută pentru cazurile în care scena generală corespunde interogării, dar subiectul solicitat lipsește. Sistemul include o metodologie de evaluare care combină analiza calitativă cu măsurători cantitative Recall@K și o interfață grafică interactivă realizată cu Gradio. Întregul pipeline rulează în regim de inferență pe un laptop obișnuit. Experimentele arată că scorul mediu de similaritate pentru top-5 crește de la 0,2647 pentru 500 de imagini la 0,3196 pentru întreaga colecție, iar Recall@1 atinge 59,16% pe un subset de 1.000 de imagini, folosind descrierile din setul de date ca interogări.

Avizul conducătorului

Conducător,



Absolvent,



Această pagină a fost lăsată goală în mod intenționat.

Content

1. Rezumat în limba română	4
1.1 Introducere	4
1.2 Stadiul actual	4
1.3 Fundamentare teoretică	5
1.4 Implementarea soluției	7
1.5 Rezultate experimentale	8
1.6 Concluzii	10
2. Work planning.....	11
3. State of the Art	12
3.1 Introduction	12
3.2 Joint embedding spaces	12
3.3 Model families in text-to-image retrieval	12
3.3.1 Dual-encoder models	12
3.3.2 Cross-encoder models	13
3.3.3 Two-stage retrieval in modern systems.....	13
3.4 CLIP and the models built on it.....	13
3.4.1 CLIP	13
3.4.2 ALIGN, BLIP, BLIP-2.....	13
3.4.3 Retrieval-augmented CLIP.....	14
3.5 Practical implementations.....	14
3.6 Datasets	14
3.7 Evaluation metrics	15
4. Theoretical fundamentals	16
4.1 Introduction	16
4.2 The Transformers architecture	16
4.2.1 Self-attention	16
4.2.2 Multi head attention.....	17
4.2.3 Positional encoding and the encoder block	17
4.3 The text encoder	19
4.3.1 From text to a sequence of tokens.....	19
4.3.2 The end-of-text token and the sentence embedding	20

4.4 The Vision Transformer	20
4.4.1 From image to a sequence of patches	20
4.4.2 The class token and positional embeddings.....	21
4.5 Contrastive pre-training.....	22
4.5.1 The training objective	22
4.5.2 The contrastive loss	22
4.6 Cosine similarity	23
4.6.1 Retrieval	24
4.6.2 Re-ranking.....	24
4.7 Text-to-image retrieval pipeline	25
4.8 Evaluation metrics	26
5. Implementation	27
5.1 Introduction	27
5.2 Technologies and development environment	27
5.2.1 Programming language and libraries	27
5.2.2 Development environment	28
5.2.3 Model loading and execution device.....	29
5.3 System architecture	29
5.3.1 Overview of the pipeline	29
5.3.2 Global configuration and parameters	30
5.4 Dataset preparation	31
5.4.1 Structure of the Flickr30k dataset	31
5.4.2 Statistical analysis.....	32
5.4.3 Integrity verification and cleaning.....	33
5.5 Image encoding	33
5.5.1 Loading and preprocessing the images.....	33
5.5.2 Batched encoding and L2 normalization	34
5.6 Query encoding and retrieval	35
5.6.1 Query encoding	35
5.6.2 Similarity search and top-K ranking	35
5.6.3 Presentation and storage of results	36
5.7 Caption-based re-ranking.....	37
5.8 Evaluation methods	38

5.8.1 Functions for the qualitative evaluation	38
5.8.2 Quantitative evaluation with Recall@K	39
5.9 Interactive demonstration.....	40
6. Experimental Results.....	41
6.1 Experimental Setup.....	41
6.2 Qualitative retrieval results	42
6.3 Effect of collection size	44
6.4 Caption verification.....	46
6.5 Caption-based re-ranking results	48
6.6 Quantitative evaluation: Recall@K	49
6.7 Interactive demonstration.....	50
6.8 Discussion of results	50
7. Conclusions	52
7.1 Summary of the work	52
7.2 Limitations and future work	52
7.3 Closing remarks.....	53
8. References	54
9. Appendix	56
9.1 Dataset preparation notebook	56
9.2 Baseline implementation notebook.....	58

1. Rezumat în limba română

1.1 Introducere

Căutarea unei imagini pornind de la o simplă descriere scrisă a devenit un gest obișnuit al vieții de zi cu zi. În spatele acestui gest se află însă o problemă complexă, numită căutare multimodală, în care sistemul trebuie să pună în legătură două tipuri diferite de date: textul scris de utilizator și conținutul vizual al imaginilor. Domeniul se situează la intersecția dintre vedere artificială și prelucrarea limbajului natural.

Obiectivul acestei lucrări este construirea unui sistem care primește o descriere textuală ca, de exemplu “un câine care se joacă în parc” și returnează, dintr-o colecție de imagini lipsite de orice etichetă manuală, imaginile care corespund cel mai bine descrierii. Sistemul realizat urmează arhitectura modernă în două etape, de tip regăsire urmată de reordonare (retrieve-then-rerank). Prima etapă folosește modelul pre-antrenat CLIP pentru a ordona întreaga colecție după similaritate, iar a doua etapă: reordonează primele rezultate pe baza descrierilor textuale asociate imaginilor. O trăsătură definitorie a lucrării este că nu se efectuează nicio antrenare, modelul este folosit exclusiv cu ponderile sale pre-antrenate, disponibile public, astfel încât întregul sistem rulează ca inferență pe un laptop obișnuit. Evaluarea se realizează pe setul de date Flickr30k, în două moduri. Calitativ, prin interogări scrise de mână și cantitativ, prin metrica standard Recall@K. Sistemul este apoi expus utilizatorului printr-o interfață grafică interactivă.

1.2 Stadiul actual

Ideea care stă la baza căutării moderne text-imagini este spațiul de reprezentare comun. Imaginile și textele sunt trecute prin rețele neuronale proprii, numite codificatoare și devin vectori în același spațiu. Acest spațiu este construit astfel încât o imagine și un text care descriu același lucru ajung aproape unul de celălalt, iar perechile fără legătură ajung îndepărtate. Odată ce ambele modalități sunt proiectate în spațiul comun, regăsirea devine simplă: interogarea este transformată într-un vector, iar imaginile ai căror vectori sunt cei mai apropiați de acesta sunt returnate ca rezultat.

Sistemele de regăsire text-imagini se împart în două familii, după locul în care reprezentările celor două modalități se combină. În modelele cu codificator dual (dual-encoder), imaginea și textul sunt procesate de două rețele separate și proiectate independent în spațiul comun. Avantajul este eficiența la căutare: fiecare imagine din colecție poate fi codificată o singură dată, în avans, iar pentru o interogare nouă este nevoie doar de o trecere prin codificatorul de text și de o căutare a celor mai apropiați vecini. În schimb, pentru că imaginea și textul nu interacționează în timpul codificării, modelul nu poate surprinde corespondențe fine între cuvinte individuale și regiuni ale imaginii. Modelele cu codificator încrucișat (cross-encoder), precum SCAN, ViLBERT, UNITER sau OSCAR, procesează perechea imagine-text ca o intrare comună și obțin o precizie superioară pe potrivirile fine. În cazul lor însă nimic nu poate fi precalculat, așa că scorarea întregii colecții la fiecare interogare este prea lentă în practică. Din acest motiv, sistemele actuale combină cele două familii într-o arhitectură în două etape: un codificator dual rapid extrage o listă scurtă de candidați din întreaga colecție, iar un al doilea model rescorează doar această listă scurtă. Lucrarea de față preia același tipar.

Modelul aflat în centrul lucrării este CLIP (Contrastive Language-Image Pre-training), un codificator dual antrenat contrastiv pe aproximativ 400 de milioane de perechi imagine-text colectate de pe internet. Pentru fiecare lot de perechi, modelul învață să asocieze fiecare imagine cu textul ei corect dintre toate combinațiile posibile, ceea ce apropie în spațiu perechile corecte și

le îndepărtează pe cele greșite. Ideea a fost apoi extinsă în mai multe direcții. ALIGN a arătat că un set de date mult mai mare, dar mai zgomotos, atinge rezultate similare, BLIP și BLIP-2 au adăugat capacitatea de a genera text, RA-CLIP îmbogățește reprezentarea imaginii consultând un set de referință de perechi imagine-text, în loc să memoreze totul la antrenare. În practică, asemenea modele sunt accesibile prin implementări deschise precum OpenCLIP și biblioteca HuggingFace Transformers. Atunci când colecția ajunge la ordinul milioanei de imagini, căutarea rămâne rapidă cu ajutorul bibliotecilor de căutare aproximativă a vecinilor, cea mai cunoscută fiind FAISS.

Pentru testare, comunitatea de cercetare folosește seturi de date standard, în care fiecare imagine este însoțită de descrieri scrise de oameni. Lucrarea de față folosește Flickr30k, cu 31,783 de imagini din scene cotidiene, fiecare descrisă independent de cinci persoane. Literatura mai folosește MS-COCO și pentru antrenarea modelelor mari, seturi colectate automat precum Conceptual Captions și LAION-400M. Rezultatele se compară prin metrica Recall@K(R@K) pentru o interogare al cărei răspuns corect este cunoscut dinainte, R@K măsoară cât de des imaginea corectă apare între primele K rezultate și se raportează de regulă pentru K egal cu 1, 5 și 10.

1.3 Fundamentare teoretică

Dificultatea centrală a căutării multimodale este că o interogare scrisă și o imagine nu pot fi comparate direct: una este o secvență de cuvinte, cealaltă o rețea de pixeli. Compararea devine posibilă abia când ambele sunt transformate în vectori care trăiesc în același spațiu. Întreaga metodă de regăsire din această lucrare se sprijină pe această idee. Un codificator de imagini și un codificator de text proiectează fiecare intrarea în același spațiu de reprezentare comun, de 512 dimensiuni. Pre-antrenarea contrastivă aranjează spațiul astfel încât o imagine și descrierea ei să ajungă aproape una de cealaltă. Similaritatea cosinus măsoară această apropiere la momentul căutării. Cele trei mecanisme sunt prezentate pe rând.

Ambele codificatoare din CLIP sunt Transformere, o arhitectură care a renunțat complet la recurență și convoluții și se bazează numai pe mecanismul de atenție. Operația centrală este auto-atenția (self-attention): fiecare element al secvenței de intrare “privește” toate celelalte elemente și decide cât contează fiecare pentru propria reprezentare. Concret, fiecare vector de intrare este proiectat într-o interogare, o cheie și o valoare. Interogarea unui element este comparată prin produs scalar cu cheile tuturor celorlalte, iar ponderile rezultate, normalizate printr-o funcție softmax, stabilesc cât din fiecare valoare intră în noua reprezentare. Operația se calculează în paralel pentru toate pozițiile, ceea ce face antrenarea mult mai rapidă decât la rețele recurente. Ea este repetată pe mai multe “capete” de atenție, fiecare putând urmări un alt tip de relație între elemente. Pentru că atenția nu are o noțiune proprie de ordine, fiecărui element se adaugă o codare de poziție, în CLIP, ambele codificatoare folosesc poziții învățate în timpul antrenării. Un bloc complet de codificator înlănțuie un strat de auto-atenție cu mai multe capete și o rețea feed-forward aplicată separat fiecărei poziții, fiecare înconjurat de o conexiune reziduală și o normalizare de strat. Codificatorul complet se obține prin stivuirea mai multor astfel de blocuri identice.

Pe latura textuală, interogarea scrisă este mai întâi segmentată de un tokenizator de tip Byte Pair Encoding (BPE), care păstrează cuvintele frecvente ca unități întregi și sparge cuvintele rare în fragmente, astfel încât orice intrare poate fi reprezentată. Secvența este completată sau tăiată la lungimea fixă de context de 77 de tokenuri și trece prin stiva de blocuri. Vectorul de la poziția tokenului de sfârșit de text, poziție care poate cuprinde prin atenție întreaga propoziție, este proiectat în spațiul comun de 512 dimensiuni și normalizat la lungimea unitară. El devine reprezentarea întregii propoziții.

Pe latura vizuală, Transformer de viziune (ViT) aplică aceeași arhitectură asupra imaginilor și schimbă doar pregătirea intrării. Imaginea, redimensionată la 224×224 de pixeli, este tăiată în patch-uri pătrate de 32×32 de pixeli, adică 49 de patch-uri, fiecare aplatizat și proiectat liniar într-un vector. În fața secvenței se adaugă un token special de clasă, care nu provine din niciun patch, ci adună prin auto-atenție informație din toate patch-urile. Secvența de 50 de tokeni trece prin cele 12 blocuri ale codificatorului, iar starea finală a tokenului de clasă, proiectată în spațiul comun și normalizată, devine reprezentarea imaginii. Configurația folosită în lucrare, ViT-B/32, a fost preferată unor variante mai mari precum ViT-L/14 pentru că secvența mai scurtă și rețeaua mai mică reduc memoria și timpul de codificare. Costul este o acuratețe ceva mai mică, acceptabil pentru dimensiunea colecției și pentru demonstrația interactivă construită aici.

Luată separat, cele două codificatoare nu au niciun motiv să plaseze o imagine lângă descrierea ei. Ceea ce leagă într-un singur spațiu multimodal este pre-antrenarea contrastivă, metodă introdusă de CLIP. Datele sunt procesate în loturi de N perechi imagine-text, iar similaritatea cosinus se calculează între fiecare imagine și fiecare text din lot. Obiectivul, o clasificare simetrică rezolvată simultan în ambele direcții, crește scorul fiecărei perechi corecte și îl scade pe al fiecărei perechi greșite. Un parametru de temperatură, învățat odată cu restul rețelei, controlează cât de puternic sunt separate perechile corecte de cele greșite, el contează doar la antrenare și nu schimbă ordinea rezultatelor la căutare. Aceasta schemă de antrenare justifică alegerile lucrării. CLIP este un codificator dual cu ponderi disponibile public, deci nu trebuie reantrenat pentru un anumit set de date, în lucrare nu se efectuează nicio antrenare, iar pre-antrenarea contrastivă este prezentată pentru că explică proprietatea pe care se sprijină întreaga metodă: perechile imagine-text care se potrivesc stau aproape în spațiul comun.

Apropierea se măsoară cu similaritatea cosinus, care compară doi vectori prin unghiul dintre ei, nu prin mărime și ia valori între -1 și 1. Toate reprezentările produse de cele două codificatoare sunt normalizate L2 la lungime unitară, așa că similaritatea cosinus se reduce la un simplu produs scalar, mai ieftin de calculat și folosit efectiv la căutare. Fluxul complet al unei interogări este atunci următorul. Într-o etapă *offline*, toate imaginile colecției sunt codificate o singură dată, iar vectorii lor sunt stocați. În etapa *online*, executată la fiecare interogare, doar textul este codificat, un singur produs matriceal dă câte un scor pentru fiecare imagine, imaginile sunt ordonate după scor, iar primele K (top- K) constituie rezultatul primei etape.

A doua etapă, opțională, este reordonarea pe baza descrierilor (re-ranking) și reia doar această listă scurtă. Interogarea este comparată, în același spațiu, cu fiecare dintre descrierile scrise de oameni ale fiecărui candidat, iar cel mai bun scor de descriere devine noul scor al candidatului. Comparăția se mută astfel între doua reprezentări de text, adică în interiorul unei singure modalități, ceea ce este de regulă mai fiabil decât potrivirea unei propoziții cu o rețea de pixeli. Reordonarea doar rearanjează candidații primei etape și nu poate recupera o imagine relevantă lăsată în afara listei scurte, dar ridică precizia primelor rezultate la un cost aproape nul.

Sistemul este evaluat în două moduri complementare. Evaluarea cantitativă folosește metrica Recall@ K și se sprijină pe structura setului de date: fiecare descriere din Flickr30k a fost scrisă pentru o imagine, așa că descrierea poate fi folosită ca interogare al cărei răspuns corect este cunoscut dinainte. Pentru fiecare astfel de interogare se verifică dacă imaginea ei apare între primele K rezultate, iar R@1, R@5 și R@10 se calculează peste toate interogările. Evaluarea calitativă folosește în schimb interogări scrise de mână, care nu au o imagine corectă fixată dinainte, deci Recall@ K nu se poate calcula pentru ele, rezultatele sunt evaluate vizual, pe baza imaginilor returnate și a scorurilor de similaritate. Cele două metode se completează: prima dă o măsură numerică, a doua arată comportamentul sistemului pe interogări formulate așa cum le-ar scrie un utilizator real.

1.4 Implementarea soluției

Implementarea urmează o abordare descendentă și este organizată în două notebook-uri Jupyter: primul pregătește și verifică setul de date, al doilea construiește sistemul de regăsire propriu zisă. Sistemul este scris în Python și folosește PyTorch pentru operațiile cu tensori și biblioteca HuggingFace Transformers pentru accesul la modelul CLIP pre-antrenat (punctul de control `openai/clip-vit-base-patch32`). La pornire, sistemul detectează automat prezența unei plăci grafice compatibile CUDA și mută modelul pe ea, modelul este încărcat direct în modul de evaluare, fiindcă este folosit numai pentru inferență. Doi parametri globali guvernează întregul sistem: `TOP_K = 5` fixează câte imagini se returnează pentru fiecare interogare, iar `BATCH_SIZE = 32` fixează câte imagini se codifică împreună într-o singură trecere.

Setul de date Flickr30k conține 31,783 de imagini JPEG și 158,915 descrieri, câte cinci pentru fiecare imagine, stocate într-un fișier text cu câmpuri separate prin caracterul “|”. Analiza statistică arată că descrierea medie are 13,39 cuvinte, cea mai scurtă 2 și cea mai lungă 82, deci aproape nicio descriere nu depășește limita de 77 de *token-i* a codificatorului de text. Înainte de utilizare, datele au fost curățate și verificate: un rând corupt, cu un separator lipsă, a fost corectat manual, copiile duplicate ale fișierelor au fost eliminate, iar două verificări automate confirmă că nu există valori lipsă și că fiecare imagine referită în fișierul de descrieri există pe disc.

Calea imaginilor este implementată de funcția `encode_images()`, partea cea mai costisitoare a sistemului. Ea încarcă imaginile de pe disc, le convertește în format RGB și le codifică în loturi de 32. O imagine coruptă este raportată și sărită dacă este cazul iar acest lucru nu oprește procesul. Codificarea rulează fără calcul de gradienti, iar fiecare vector rezultat este normalizat L2, astfel încât similaritatea cosinus calculată la căutare se reduce la un produs scalar. Rezultatul este o matrice de dimensiune $N \times 512$, câte un vector pentru fiecare imagine încărcată, împreună cu lista căilor de fișier corespunzătoare.

Calea interogării este implementată de funcția `search()`. Textul este *tokenizat*, trecut prin codificatorului de text și normalizat la fel ca imaginile, apoi întreaga comparație se face printr-o singură înmulțire de matrice, care produce câte un scor pentru fiecare imagine din colecție. Primele K imagini după scor formează rezultatul primei etape, iar fiecare rezultat păstrează rangul, numele fișierului și scorul de similaritate. Peste `search()` sunt construite funcțiile de evaluare: una rulează un set întreg de interogări pe aceleași reprezentări, codificate o singură dată, iar rezultatele se salvează în fișierele CSV pentru reproducere și comparație.

Reordonarea pe baza descrierilor este implementată de funcția `rerank_with_captions()`. Ea obține mai întâi lista scurtă de candidați prin `search()`, codifică interogarea o singură dată, apoi caută în setul de date descrierile fiecărui candidat, le codifică și calculează similaritatea dintre interogare și fiecare descriere, cel mai bun scor de descriere devine noul scor al candidatului, iar lista este reordonată. Funcția returnează atât ordinea inițială, cât și cea reordonată, pentru ca efectul reordonării să poată fi analizat. La compararea lor se urmăresc numai schimbările de poziție, nu valorile scorurilor: scorul primei etape măsoară o similaritate text-imagine, iar scorul reordonării o similaritate text-text, deci cele două nu se află pe aceeași scală și nu pot fi comparate numeric.

Evaluarea cantitativă este implementată de funcția `evaluate_recall()`, care folosește fiecare descriere drept interogare și verifică dacă imaginea ei apare între primele K rezultate. `Recall@K` se raportează numai pentru prima etapă, nu și pentru reordonare: interogarea fiind ea însăși o descriere din set, reordonarea ar compara-o cu propriile descrieri ale imaginii corecte, iar potrivirea trivială ar umfla scorul. În sfârșit, sistemul este expus printr-o interfață grafică construită cu Gradio,

în care utilizatorul scrie o interogare într-un browser, se alege dimensiunea subsetului căutat, între 100 și toate cele 31,783 imagini cu valoare implicită de 5,000 și numărul de rezultate afișate K, între 1 și 5.

1.5 Rezultate experimentale

Toate experimentele rulează pe un singur laptop, cu procesor AMD Ryzen 7 7435HS (8 nuclee, 16 fire de execuție), 16 GB de memorie RAM și o placă grafică NVIDIA GeForce RTX 4060 cu 8 GB, pe care rulează toată codificarea de imagini și text, prin CUDA. Fiecare interogare returnează primele 5 imagini, iar imaginile se codifică în loturi de 32. Evaluarea calitativă folosește opt interogări scrise de mână pentru această lucrare, nu preluate dintre descrierile setului de date, alese să acopere scene variate: oameni în grup și singuri, animale, obiecte, scene urbane și un peisaj. Fiecare interogare este rulată pe cinci subseturi, de la 500 de imagini până la toată colecția de 31,783. Un subset de dimensiune N conține primele N imagini din listă, deci orice imagine dintr-un subset mic există și în cele mari. Datorită acestui lucru, un rezultat poate părăsi topul doar dacă o imagine nou adăugată a obținut un scor mai mare.

Primul experiment judecă direct calitatea imaginilor returnate, pe colecția completă. Scorurile primului rezultat se întind între 0.271 și 0.361 pe cele opt interogări. Cea mai bună interogare “a woman sitting at a cafe table with a laptop and a cup of coffee”, atinge 0.361 cu o imagine care satisface fiecare element al propoziției, iar scorurile următoarelor rezultate scad treptat, pe măsură ce imaginile pierd câte un element al descrierii: 0.351, 0.342, 0.332, 0.322. Cea mai dificilă interogare, “sunset over a mountain range with a lake in the foreground”, rămâne între 0.271 și 0.259, iar cauza este colecția, nu modelul. Flickr30k conține scene cotidiene centrate pe oameni și aproape niciun peisaj pur, așa că sistemul așază în top cele mai bune imagini disponibile, care combină doar parțial apusul, muntele și lacul. Un scor mic este deci un semnal despre colecție, nu un eșec al căutării. Interogarea cu câinele arată în schimb limita reală a primei etape: primul rezultat, cu scorul 0.350, este un parc cu o bancă și un copil, fără niciun câine. Imaginea se potrivește cu scena interogării, dar îi lipsește subiectul, fiindcă întreaga imagine și întreaga propoziție sunt comprimate fiecare într-un singur vector, iar o potrivire puternică pe ansamblul scenei poate cântări mai mult decât un obiect lipsă.

Al doilea experiment măsoară efectul dimensiunii colecției. Tabelul 1 arată scorul mediu al primelor 5 rezultate, mediat peste cele opt interogări, pentru fiecare subset.

Tabelul 1. Scorul mediu al primelor 5 rezultate în funcție de dimensiunea colecției.

	500	5,000	10,000	20,000	31,783
Scor mediu top-5	0.264667	0.298134	0.310668	0.315291	0.319608
Creștere față de subsetul anterior		+0.033467	+0.012534	+0.004623	+0.004317

Câștigurile scad de la un pas la altul. Primul salt, de la 500 la 5,000 de imagini, aduce +0.033467, ultimul de la 20,000 la colecția completă, doar +0.004317. Explicația este simplă: odată ce colecția conține o potrivire bună, o imagine nouă intră în top 5 numai dacă depășește imaginile deja găsite, iar cu cât rezultate curente sunt mai bune, cu atât mai puține imagini le pot depăși. Suprapunerea listelor confirmă același proces din altă direcție. Între subseturile de 500 și 5,000 de imagini supraviețuiesc doar 2 din cele 40 de poziții de top-5, între subseturile de 10,000 și 20,000 persistă 28 din 40, iar la pasul final 25 din 40. Pentru cele opt interogări de test, atât scorurile, cât și componența listelor se stabilizează în jurul a 20,000 de imagini.

Al treilea experiment verifică judecățile vizuale față de o sursă independentă, cele cinci descrieri umane care însoțesc fiecare imagine din Flickr30k, scrise de adnotatori care nu cunoșteau nicio interogare. Verificarea confirmă evaluarea făcută cu ochiul liber. Pentru interogarea cu câinele, imaginea clasată a doua este descrisă de adnotatori aproape în cuvintele interogării (“A dog chases a toy in the grass with its owner in the background .”) în timp ce niciuna dintre descrierile imaginii clasate prima nu menționează vreun câine. Verificarea și rafinează evaluarea, a doua imagine de la interogarea cafenelei, care vizual trecea drept o femeie cu laptop, la cafenea, este descrisă de adnotatori ca un birou întunecat, deci potrivirea este mai slabă decât părea. Aceeași verificare scoate la iveală și zgomotul adnotărilor, o descriere la persoana întâi care nu descrie nimic din imagine. Acest zgomot contează pentru etapa următoare, fiindcă exact aceste descrieri devin materialul reordonării, luarea maximului peste cele cinci descrieri o face tolerantă la o singură descriere proastă.

Al patrulea experiment evaluează reordonarea pe baza descrierilor, aplicată pe subșetul de 5,000 de imagini. Pe acest subșet, prima etapă face pentru interogarea cu câinele aceeași eroare ca pe colecția completă, primul loc este ocupat de o scenă din parc fără câine. Reordonarea o trimite pe ultimul loc din cinci, iar pe primul loc urcă o imagine descrisă de adnotatori ca “a woman sitting on a bench with a latte in her lap and a white dog on a blue leash to her side .”, aproape interogarea însăși. Interogarea cu balonul se comportă la fel, imaginea de pe primul loc, fără balon după descrierile ei, coboară pe locul al treilea, iar o imagine descrisă ca “A child holding a helium filled balloon” urcă pe primul loc. Interogarea cafenelei arată cazul complementar, primele două rezultate erau deja corecte și reordonarea le lasă neatinse, rearanjând doar jumătatea de jos a listei. Comportamentul este exact cel așteptat de la un sistem în doua etape: etapa vizuală rapidă greșește ocazional punând scena deasupra subiectului, iar etapa pe descrieri repară tocmai aceste cazuri și nu strică rezultatele corecte, la un cost neglijabil, codificând 25 de descrieri în loc de întreaga colecție. O precizare este obligatorie la citirea acestor rezultate: scorurile primei etape (între 0.29 și 0.35) și scorurile reordonării (între 0.62 și 0.86) nu se compară numeric, fiindcă primele măsoară similaritate text-imagine, iar celelalte similaritate text-text. Informația este schimbarea poziției.

Al cincilea experiment oferă o măsură numerică, prin Recall@K pe subșet-uri. Toate descrierile imaginilor din colecție servesc drept interogări, câte cinci pentru fiecare imagine, iar o interogare contează ca reușită la K dacă propria ei imagine apare între primele K rezultate. Tabelul 2 arată rezultatele.

Tabelul 2. Recall@K pentru regăsirea text-imagine, cu descrierile drept interogări.

Dimensiunea colecției	Interogări	R@1	R@5	R@10
1,000	5,000	59,16%	84.50%	90.66%
5,000	25,000	40,33%	66.66%	76.20%
20,000	100,000	25.36%	47.17%	56.85%

Recall@K scade pe măsură ce colecția crește, fiindcă fiecare imagine adăugată este un distractor în plus. Două repere ajută la citirea valorilor. Față de o ordonare aleatorie, care ar plasa imaginea corectă pe primul loc în 0,1% din cazuri la 1,000 de imagini și în 0,005% la 20,000 de imagini, sistemul atinge 59,16% și respectiv 25,36%. În schimb, comparația cu valorile Recall@K publicate pentru CLIP pe Flickr30k trebuie evitată. Acelea sunt măsurate pe partiția standard de test de 1,000 de imagini fixe, pe când subșeturile de aici sunt pur și simplu primele N imagini ale colecției, iar tabelul 2 arată el însuși cât de mult depinde Recall@K de dimensiunea mulțimii căutate. Valorile de aici sunt măsurători interne și pot servi drept referință dacă o componentă a sistemului, de exemplu codificatorul, este înlocuită.

Dincolo de experimentele cu scenariu fix, sistemul funcționează și interactiv, prin interfața grafică. O interogare nouă care nu face parte dintre cele opt teste, este tratată la fel ca în restul capitolului. Timpul de răspuns este dominat de codificarea subsetului ales, pe care o interfața o reia la fiecare interogare. Pe mașina de test, o interogare pe 5,000 de imagini durează aproximativ 32 de secunde. Aproape tot acest timp se consumă la recodificarea imaginilor, fiindcă interfața reia codificarea subsetului la fiecare interogare. Căutarea propriu zisă înseamnă doar o codificare de text și o înmulțire de matrice, deci contribuie neglijabil la total. Pentru o demonstrație, un asemenea timp de răspuns este acceptabil.

1.6 Concluzii

Lucrarea tratează căutarea multimodală în forma regăsirii imaginilor pe baza descrierilor textuale. Utilizatorul descrie imaginea dorită într-o propoziție scurtă, iar sistemul o caută într-o colecție fără nicio etichetă manuală. Prima etapă folosește codificatorul dual CLIP ViT-B/32 pre-antrenat, care proiectează interogarea și cele 31,783 de imagini din Flickr30k în același spațiu de reprezentare de 512 dimensiuni și ordonează imaginile după similaritatea cosinus. A doua etapă reordonează primii candidați după descrierile lor umane. Nu se antrenează niciun model, totul rulează ca inferență pe un singur laptop și este accesibil printr-o interfață grafică.

Experimentele conduc la patru concluzii. Scorurile absolute ale primei etape sunt modeste, între 0.271 și 0.361 pentru cel mai bun rezultat al fiecărei interogări de test, dar acest nivel ține de antrenare contrastivă, care învață modelul să claseze perechea potrivită deasupra celor nepotrivite, nu să împingă similaritatea spre 1. Informația stă în ordine, iar ordinea s-a dovedit de încredere în fiecare experiment care a testat-o. Singurul eșec sistematic al primei etape este potrivirea scenei fără subiectul cerut, iar reordonarea repară exact aceste cazuri la un cost neglijabil, fără să strice rezultatele deja corecte. Pe măsura ce colecția crește, rezultatele se stabilizează: scorul mediu al primelor 5 rezultate urcă de la 0.264667 pe 500 de imagini la 0.319608 pe colecția completă, iar de la aproximativ 20,000 de imagini componența topului practic nu se mai schimbă. Numeric cu descrierile drept interogări, $R@1$ este 59,16% pe 1,000 de imagini și scade la 25,36% pe 20,000 de imagini, fiindcă fiecare imagine adăugată este un distractor în plus. Aceste valori sunt referințe interne și nu se compară cu rezultatele publicate pe Flickr30k, măsurate pe partiția standard de test.

Sistemul are două limitări. Reprezentările imaginilor nu sunt păstrate între rulări, așa că interfața recodifică subsetul ales la fiecare interogare, aici se consumă aproape toate cele aproximativ 32 de secunde pe interogare la 5,000 de imagini, iar stocarea reprezentărilor calculate o singură dată ar reduce timpul de răspuns la codificarea interogării. A doua limitare este codificatorul: ViT-B/32 este cea mai mică configurație de Vision Transformer a lui CLIP, aleasă pentru viteză, iar înlocuirea ei cu ViT-L/14, care procesează imaginea în *patch*-uri de 14×14 pixeli în loc de 32×32 , ar ridica probabil atât scorurile de similaritate, cât și valorile Recall@K, cu prețul unui timp de codificare mai lung. Lucrarea poate fi extinsă în trei direcții: căutarea aproximativă a vecinilor, de exemplu cu FAISS, ar păstra viteza dacă colecția ar crește mult peste dimensiunea actuală, ajustarea fină a lui CLIP pe Flickr30k ar adapta reprezentările generaliste la scenele cotidiene ale acestui set, în schimbul antrenării pe care implementarea actuală o evită intenționat, iar repetarea evaluării Recall@K pe partiția standard de test de 1,000 de imagini ar face rezultatele direct comparabile cu literatura.

În ansamblu, lucrarea arată că un sistem funcțional de căutare multimodală se poate construi dintr-un codificator dual pre-antrenat, similaritatea cosinus și descrierile deja existente în setul de date, fără a antrena vreun model. Spațiul de reprezentare pre-antrenat conține suficientă structură semantică pentru ca ordonarea după similaritate să returneze imagini plauzibile pentru text arbitrar dintre peste treizeci de mii de candidați, iar cele două etape, regăsirea vizuală rapidă și reordonarea mai precisă, reproduc la scară mică arhitectura sistemelor moderne de regăsire.

2. Work planning

Figure 1 presents the timeline and distribution of work over the past months for the thesis. The tasks are organized starting chronologically with documentation first and with the implementation part after.

ID	Task Name	Start	End
1	Thesis	2025-10-28	2026-07-06
2	Choosing the theme	2025-10-28	2025-10-30
3	Documentation: neural networks & NLIP fundamentals	2025-11-03	2026-03-11
4	Receiving reference materials Flickr30k dataset + RA-CLIP paper	2025-11-18	2025-11-18
5	Study of Transformers, ViT, CLIP and contrastive learning	2025-11-18	2026-01-21
6	Development environment setup (Python, Pytorch, venv, VS Code)	2026-03-20	2026-03-30
7	Dataset download, exploration and integrity verification	2026-03-23	2026-03-31
8	Baseline CLIP pipeline: subset loading, encoding, similarity search, top-K + query testing	2026-04-02	2026-05-01
9	Evaluation over growing subsets, re-ranking, Recall@K and GUI implementation	2026-05-15	2026-06-22
10	Thesis writing	2026-05-29	2026-07-06

Figure 1. Table of work planning

3. State of the Art

3.1 Introduction

Searching for an image by simply typing a short description has become a normal thing in our everyday life. Behind the simple action there is a more complex problem, called multimodal search, where the system must connect two different types of data: text written by the user and the visual content of the images. This field sits between two big domains, computer vision and natural language processing.

The purpose of this thesis is to build a system that takes a description (a query, for example: “a dog playing in a park”) and returns the images that best match the query from a collection, without those images having manually written tags. At the center of this work is the CLIP model.

3.2 Joint embedding spaces

The idea that makes modern text to image search work is the joint embedding space. The images and the text are passed through their own neural networks, called encoders, and they end up as vectors in the same space. The space is built so that an image and a text that describe the same thing land close to each other, while unrelated pairs land far apart. Once both modalities have been projected into the shared space, text-to-image retrieval becomes simple: the sentence is turned into a vector, and the images whose vectors are closest to it are returned as a result.

The image encoder is usually a Vision Transformer [1], which cuts the image into small patches and treats them as a sequence, and the text encoder is a Transformer [2] as well, the same architecture used in modern language models. The method that these two encoders are trained so that their outputs end up in a shared space is called contrastive learning, and the model that made this approach popular is CLIP [3].

3.3 Model families in text-to-image retrieval

Text-to-image retrieval systems fall into two broad model families, distinguished by where in the pipeline image and text representations are combined. That architectural choice drives the core trade-off in any practical system: how fast it can search versus how accurately it can match.

3.3.1 Dual-encoder models

In a dual-encoder (also called a bi-encoder), the image and the text are processed by two separate networks and projected independently into a shared vector space, where similarity is computed as a dot product. CLIP belongs to this family, as do the embedding methods that preceded it. The earliest representative is VSE++ [4], which encodes images with a convolutional image encoder and text with a recurrent encoder. Its main contribution was a change to the training objective: rather than averaging the loss over all negative pairs in a batch, it focuses only on the hardest negative for each anchor, the one most easily confused with the correct match. This hard negative mining strategy remains standard in contrastive training today, including in CLIP-style models.

The main advantage of dual-encoders is search-time efficiency. Because the two modalities are encoded separately, every image in a collection can be embedded once in advance and stored in an

index. A new text query then requires only a single pass through the text encoder followed by a nearest-neighbour search, which is what makes large-scale retrieval practical. This is why dual-encoders like CLIP are the standard first stage in modern retrieval systems. The trade-off is that, because image and text never interact during encoding, the model cannot capture fine-grained correspondences between specific words and specific image regions.

3.3.2 Cross-encoder models

A cross-encoder takes an image-text pair as a single joint input and produces one relevance score, letting the two modalities interact directly through attention. That direct interaction is what gives cross-encoders their accuracy advantage on fine-grained matching. An early example is SCAN [5], which detects a set of regions in the image and computes attention between every region and every word in the sentence, then aggregates these local alignments into the final similarity score rather than relying on a single global comparison. Later models scaled this idea through large-scale pre-training: ViLBERT [6] extends the BERT architecture into two interacting streams, one for image regions and one for text tokens, joined by co-attentional layers; UNITER [7] pre-trains a single joint transformer over several image-text datasets and adds an explicit word-region alignment objective. OSCAR [8] follows the same joint-encoding approach but simplifies alignment by feeding the names of detected objects into the model as anchor points shared between the image and the caption.

The drawback of cross-encoders is the mirror image of the dual-encoder’s strength: because image and text must be processed together, nothing can be precomputed, and scoring every image in a large collection against each query is too slow to be practical on its own.

3.3.3 Two-stage retrieval in modern systems

Because each family has a complementary strength and weakness, current systems combine them in a two-stage pipeline rather than choosing one over the other. In the first stage, a fast dual-encoder such as CLIP retrieves a short list of candidate images from the full collection using indexed nearest-neighbour search. In the second stage, a separate model rescores only that short list, where higher accuracy can be applied without the cost of running it over every image. This retrieve-then-rerank design is the standard architecture in modern retrieval: the first stage handles scale, the second handles precision.

3.4 CLIP and the models built on it

3.4.1 CLIP

CLIP (Contrastive Language-Image Pre-training) [3] is the model used as a base in this thesis. It has the dual encoder structure described above and was trained on around 400 million image-text pairs collected from the internet. The training is contrastive: for a batch of N pairs, the model has to match each image with its correct text out of all the possible combinations, pulling the correct pairs together and pushing the wrong ones apart.

3.4.2 ALIGN, BLIP, BLIP-2

Several models extended CLIP in different directions. ALIGN [9] showed that a much larger but noisier dataset, with almost no cleaning, can reach similar results, which lowers the cost of preparing the data. BLIP [10] and later BLIP-2 [11] added the ability to also generate text, so the same model can do both, searching images and describing them or answering questions about them by using large pre-trained language models.

3.4.3 Retrieval-augmented CLIP

The model most directly connected to this thesis is RA-CLIP (Retrieval Augmented CLIP) [12]. The authors point out that normal CLIP must memorize all visual concepts during training, which is why it needs so much data. Instead, they keep a part of the image-text pairs aside as a reference set, and for an input image they retrieve the most similar pairs from this set and use them to enrich the image representation. The reference set acts as an external source of knowledge that the model can consult instead of memorizing everything, and with this the authors get better results than plain CLIP using the same amount of data.

3.5 Practical implementations

Besides the models themselves, an important part is the set of tools that make these models usable in practice. The first one is OpenCLIP [13], an open-source implementation of CLIP that closely reproduces the original results and offers many pretrained models trained on public datasets such as LAION-400M and LAION-2B. While the original CLIP was trained by OpenAI on a private dataset, OpenCLIP makes the same kind of model freely available, and it is one of the most common ways CLIP is used today. Another widely used implementation is the HuggingFace Transformers library [14], which exposes CLIP through a unified Python API.

A second important tool appears when the image collection becomes large. Comparing the query vector with every image one by one works for a few thousand images, but it becomes slow when there are millions of them. For this reason, specialized libraries for fast similarity search are used, the best known being FAISS (Facebook AI Similarity Search) [15]. FAISS builds an index over the image vectors so that the closest ones to a query can be found very quickly, even for collections of billions of vectors, and it supports cosine similarity directly by working on normalized vectors. Tools like FAISS are what allow multimodal search systems to stay fast as the collection grows.

3.6 Datasets

To train and test systems, the research community uses a number of standard datasets, where each image comes together with one or more captions written by people. Over the years these datasets grew a lot in size, from a few thousand carefully described images to hundreds of millions of pairs collected automatically from the internet, and the choice between them usually depends on whether the goal is to train a model from scratch or only to evaluate it.

The most classic datasets for image-text matching are Flickr30k and MS-COCO, and they are still used today as benchmarks. Flickr30k [16] contains 31,783 images of everyday scenes, each one described by five short captions written by different people through crowdsourcing. MS-COCO [17] is built in a similar way but is larger, with around 123,000 images, also with five human captions each, and it focuses on common objects placed in normal, everyday situations. Because both datasets give several captions per image, they show that there is no single way to describe a

picture, and their relatively small size makes them convenient for testing a model and reporting results.

For training large models like CLIP, much bigger datasets are needed, and these are usually collected automatically rather than written by hand. Conceptual Captions [18] is one example with around 3.3 million image-text pairs where the captions come from alt text of web images after some cleaning. Even larger is LAION-400M [19], with 400 million image-text pairs gathered from the internet and filtered with CLIP itself, together with its bigger version LAION-2B used to train OpenCLIP. These datasets are much noisier than Flickr30k or MS-COCO, because the captions are not checked by people, but their huge size is what allows the models to learn so many different concepts.

3.7 Evaluation metrics

In the text-to-image retrieval literature, results are compared using a metric called Recall at K, written $R@K$. The idea is simple: for a given query, all items in the collection are sorted by similarity score, and $R@K$ counts how often the ground truth match appears within the top K positions. Because Flickr30k and MS-COCO each pair every image with five independently written captions, the evaluation is typically carried out in both directions separately. The process is as follows: given a caption, retrieve the matching image, and given an image, retrieve one of its matching captions. Both directions are reported at K equal to 1, 5, and 10. $R@1$ is the most demanding of the three, requiring the correct result to be ranked first, while $R@5$ and $R@10$ accept it anywhere in the top five or ten positions. Together, these six numbers $R@1$, $R@5$ and $R@10$ for each retrieval direction provide a complete picture of system performance, covering both strict retrieval performance ($R@1$) and more lenient retrieval performance ($R@10$).

4. Theoretical fundamentals

4.1 Introduction

This chapter explains the technical mechanisms that make multimodal search systems work. At the center of this is the idea of a shared embedding space. A written query and an image are two different kinds of data that cannot be compared directly: one is a sequence of words, the other a grid of pixels. Multimodal search becomes possible only when both are turned into vectors that live in the same space, where each image is represented by a well-defined embedding. The whole retrieval method in this thesis rests on this single idea: an image encoder and a text encoder each map their input into the same 512-dimensional space, contrastive pre-training arranges that space so that an image and its description fall close together, and cosine similarity measures that closeness at search time. The three mechanisms presented in this chapter are the means by which this shared space is built and then queried. The retrieval pipeline built here relies on three main ideas: the Transformers architecture, which is the basis of both the image and the text encoder, contrastive pre-training, which is what places images and their descriptions into a shared vector space, and the use of cosine similarity as a way to measure how close a query is to an image. Each of these is presented below. The aim is to describe how these components operate, since they are the components directly used in the proposed system.

4.2 The Transformers architecture

Both encoders inside CLIP are Transformers, so the Transformer is the starting point for understanding the model. The Transformer was introduced as an architecture for machine translation that removed recurrence and convolutions completely and relied only on a mechanism called attention [2]. Its main advantage over older recurrent networks is that it processes all positions of an input sequence in parallel, instead of one step at a time, which makes it faster to train and better at capturing relations between distant elements of a sequence. This ability to relate distant elements while processing the sequence in parallel is the reason the Transformer replaced recurrent and convolutional encoders for this kind of task, and it is why the adopted model, CLIP, builds both of its encoders on this architecture rather than on older ones [2], [3].

4.2.1 Self-attention

The central operation of the Transformer is self-attention. The concept relies on the fact that every element of the input sequence is allowed to look at every other element and decide how much each of them matters for its own representation. To do this, each input vector is projected into three different vectors using three learned weight matrices: a query, a key and a value. The query of one element is compared against the keys of all elements, and the result of this comparison decides how much each value is mixed into the output.

The comparison is done with a dot product between the query and every key, the result is scaled, passed through a softmax function so that the weights become positive and sum to one, and finally used to compute a weighted sum of the value vectors. If the queries, keys and values are grouped into matrices Q , K and V , the operation can be written compactly as in equation (1).

$$Attention(Q, K, V) = softmax\left(\frac{(Q \cdot K^T)}{\sqrt{d_k}}\right) \cdot V \quad (1)$$

In equation (1) d_k is the dimension of the key vectors, and the division by its square root is a scaling factor that keeps dot products from growing too large, which would otherwise push the softmax into regions where its gradient is very small [2]. The softmax is applied row by row, so each element ends up with a set of attention weights over all the other elements, and its new representation is a weighted average of the values according to those weights.

To make this concrete, consider a sentence of five words. Each word is first converted into a vector by a learned embedding, then projected into a query, a key and a value. The query of one element is compared against the keys of all five words, producing five raw scores. After scaling and applying softmax, these become five weights that sum to one, and the output for that element is a weighted mixture of all five value vectors. The model then learns which elements should attend to which others entirely from data, without any manually specified rules [2].

The attention operation is computed in parallel for all positions simultaneously. The entire sequence of queries is stacked into the matrix Q , all keys into K and all values into V , so equation (1) gives the output for all positions in one matrix multiplication. This parallelism is the main reason Transformers can be trained significantly faster than recurrent architectures, where the hidden state must be updated one step at a time before the next can begin [2]. A consequence of using dot product attention is its sensitivity to the magnitude of the scores. When d_k is large, the dot products can become very large, which concentrates the softmax distribution sharply and produces near zero gradients. The scaling factor $\frac{1}{\sqrt{d_k}}$ was introduced to prevent this and keep training numerically stable [2].

4.2.2 Multi head attention

Using a single attention operation forces the model to mix all types of relations into one representation. To avoid this, the Transformer uses several attention operations in parallel, called heads. Each head has its own query, key and value matrices, so each one can focus on a different kind of relation between elements. The outputs of all heads are concatenated and passed through one more linear projection, as shown in equation (2):

$$\begin{aligned} MultiHead(Q, K, V) &= Concat(head_1, \dots, head_h) \cdot W^O \\ \text{where } head_i &= Attention(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \end{aligned} \quad (2)$$

In equation (2), the weight matrices (W) are the learned projections of each head, while W^O is the projection applied after concatenation. A single attention operation would have to compress all types of inter element relationships into one weight distribution, which is limiting. By using separate heads, the model can interpret different types of relations in parallel and combine them through the final projection [2]. In the original Transformer, Vaswani et al. [2] used 8 heads and set the dimension of each head to $\frac{d_{model}}{head} = 64$, so that the total computational cost remained comparable to a single full dimension attention operation [2].

4.2.3 Positional encoding and the encoder block

Unlike recurrent networks, which process tokens one at a time and naturally maintain a sense of order, the attention operation described in the previous sections has no built in awareness of position. Because every position attends to every other position through the same mechanism, the

model cannot distinguish where in the sequence an element appears. Adding a position encoding to each input vector solves this. The original Transformer uses fixed sinusoidal functions of different frequencies, one value per embedding dimension, defined as in equation (3):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right), PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \quad (3)$$

In equation (3) d_{model} is the dimension of the embeddings. Each dimension of the encoding corresponds to a sinusoid of a different wavelength, which gives every position a unique pattern, while still letting the model generalize to sequence lengths it has not seen during training [2]. This encoding has the same size as the input embeddings, so the two can simply be added together. It is worth noting that CLIP does not use this fixed sinusoidal form, both its text encoder and its image encoder instead use learned positional embeddings, that are position vectors optimized during training rather than fixed in advance [3].

Once positions are encoded, the data passes through a stack of identical encoder blocks, whose overall structure is illustrated in Figure 1. Each block is made of two sub-layers: a multi-head self-attention layer and a position-wise feed-forward network. The feed-forward network is applied to each position separately and consists of two linear transformations with a non-linearity between them, as shown in equation (4):

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (4)$$

In equation (4) W_1, W_2 and the biases b_1, b_2 are learned parameters, and $\max(0, xW_1 + b_1)$ is ReLU activation.

Self-attention mixes information between positions, while the feed-forward network processes each position on its own, which lets the model transform the representation gathered by attention.

Around each of the two sub-layers there is a residual connection followed by layer normalization, so the output of a sublayer is $\text{LayerNorm}(x + \text{Sublayer}(x))$ [2]. The residual connection adds the input of the sub-layer back to its output, which helps the gradient flow through the network and makes it possible to stack many layers without the training becoming unstable. Layer normalization keeps the values in each layer in a stable range, which speeds up and stabilizes training.

Putting these components together, a single encoder block consists of two sub-layers stacked in sequence: a multi-head self-attention sub-layer followed by a position-wise feed-forward network. Each sub-layer is wrapped by a residual connection and a layer normalization step, as described above. The input first passes through self-attention, where every position attends to all others in the sequence, and the result is added back to the input and normalized. This output then enters the feed-forward network, which is again followed by a residual connection and normalization. The full encoder is built by stacking N such blocks on top of each other. Figure 1 illustrates this structure, showing how the input embeddings and positional encodings flow through the attention and feed-forward sub-layers, together with the residual paths that connect each sub-layer’s input to its output.

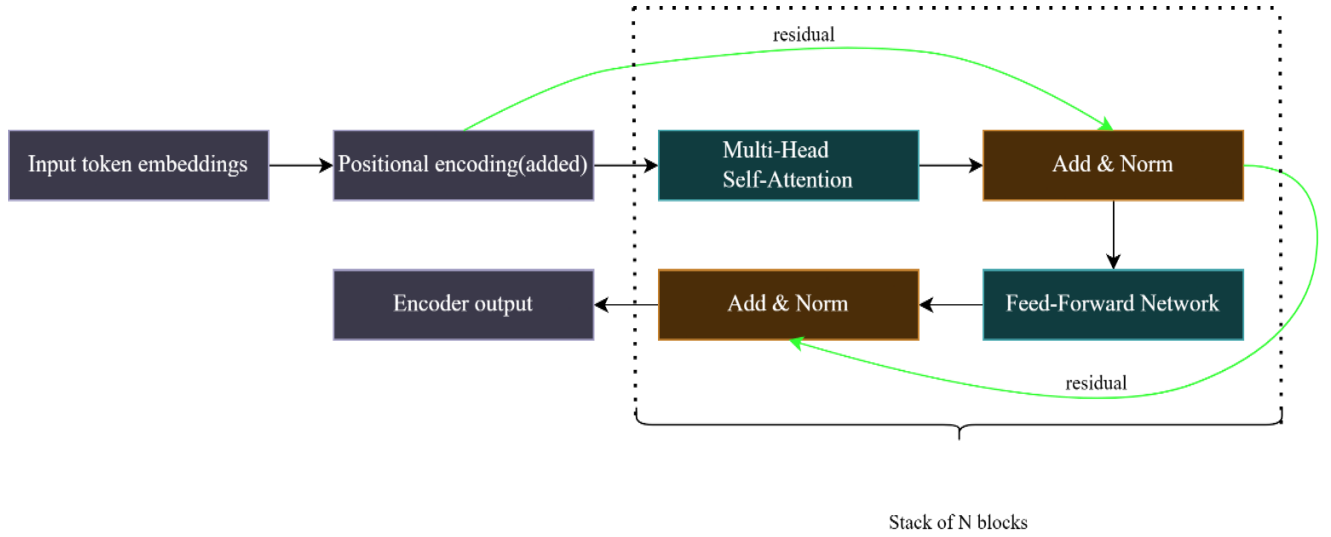


Figure 1. The Transformer encoder block.

By stacking several such blocks, the encoder builds more abstract representations of the input sequence: the lower layers capture local relations between nearby elements, and the higher layers combine them into a representation of the whole sequence. The original Transformer used six stacked blocks in both the image encoder and the text encoder. The same general encoder structure is therefore used for both modalities: text is processed as a sequence of tokens, while images are processed as a sequence of patches.

4.3 The text encoder

The text side of CLIP reuses the Transformer encoder of Section 4.2 without changing it. What is specific to the text modality is only how the written query is prepared before it enters the encoder, and how a single vector is taken out of the encoder at the end to represent the whole sentence. These two steps are described below, and they are the text counterpart of the patch preparation and class token used on the image side.

4.3.1 From text to a sequence of tokens

The encoder presented before works on a sequence of vectors, but a written query does not arrive in that form, it arrives as a string of characters. Before the encoder can process it, the sentence has to be cut into units and turned into vectors, in the same way that the image is cut into patches in Section 4.4. CLIP does this with a tokenizer based on Byte Pair Encoding (BPE), a scheme that builds its vocabulary from the most frequent character sequences found in the training text [3]. Instead of splitting on whole words, BPE keeps common words as single tokens and breaks rarer or unseen words into smaller fragments, so that any input can be represented even when it contains a word the model has never met. Each token in this fixed vocabulary has an index, and the sentence becomes a list of such indices.

Two special tokens are added around the sentence, one marking the start of the text and one marking its end. The sequence is then either padded or cut so that its length matches a fixed value: in CLIP this context length is 77 tokens, which means a query longer than that is truncated and only its first part is kept [3]. This is also the maximum sequence length enforced in the implementation developed in this thesis when the query and the captions are tokenized, so that

every input handed to the text encoder has the same length. Each index is then mapped to a learned token embedding, and a learned positional embedding is added to it, giving the sequence of vectors that the encoder blocks expect.

4.3.2 The end-of-text token and the sentence embedding

After the sequence passes through the stack of encoder blocks, a single vector has to be taken out of it to represent the whole sentence, just as the class token represents the whole image on the visual side. CLIP does not use a separate class token here, instead it takes the output at the position of the end-of-text token, since that position is allowed to attend to every word before it and therefore summarizes the full sentence [3]. This vector is projected into the same 512-dimensional shared space as the image vector and normalized to unit length, after which the two can be compared directly with the cosine similarity.

4.4 The Vision Transformer

The text encoder described in the previous section is a Transformer, but the same architecture is also used for the image side of CLIP. This is possible because of the Vision Transformer (ViT) [1], which adapts the Transformer to images without changing the encoder itself. The only new idea is the way the image is prepared: instead of feeding in a grid of pixels, the image is turned into a sequence, so the self-attention mechanism can be applied to it directly. The image encoder used in this thesis, ViT-B/32, follows this approach. A Vision Transformer was adopted for the image side, rather than a convolutional encoder, because it reuses the same self-attention machinery as the text encoder and because the strongest image encoders distributed with CLIP are of this type [1],[3].

4.4.1 From image to a sequence of patches

A transformer works on a sequence of vectors, so the first step is to convert the image into such a sequence. The image is first resized to a fixed size of 224×224 pixels and then cut into non-overlapping square patches of 32×32 pixels. At this size the image is divided into 49 patches, since the width and the height each contain $224/32 = 7$ patches. The number of patches in general is given by equation (5):

$$N = \frac{(H \times W)}{P^2} \quad (5)$$

In equation (5) H and W are the height and width of the image and P is the patch size.

Each patch is then flattened into a vector and passed through a single linear projection that maps it into the dimension used inside the encoder, as shown in Figure 2. The result is a sequence of vectors called patch embeddings. They play the same role for the image that word embeddings play for text: they are the tokens that the Transformer will process [1].

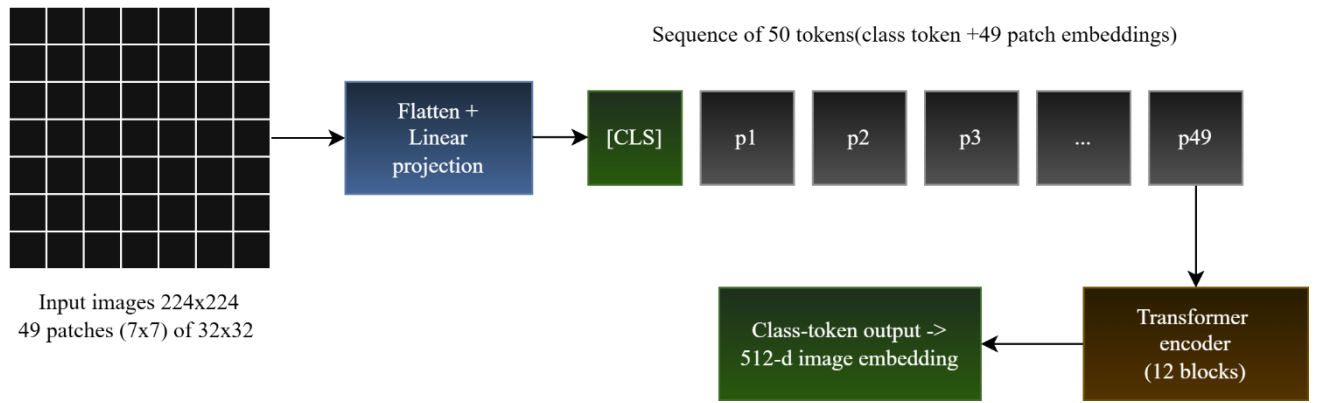


Figure 2. Image-encoding pipeline of ViT-B/32.

4.4.2 The class token and positional embeddings

Two more elements are added before the sequence enters the encoder. First, a special learnable vector called the class token ([CLS]) is placed at the beginning of the sequence. This token does not come from any patch. Its purpose is to collect information from all patches through self-attention, so that after the encoder its final state can be used as a single representation of the whole image [1]. Second, a positional embedding is added to every token, including the class token, so the model knows the original position of each patch in the image. As mentioned earlier, the Vision Transformer uses learned positional embeddings rather than the fixed sinusoidal form, that is, position vectors that are optimized together with the rest of the network [1].

The complete sequence, made of the class token followed by the patch embeddings, is then processed by the same Transformer encoder described earlier: a stack of blocks, each with multi-head self-attention and a feed-forward sub-layer, residual connections and layer normalization. After the last block, only the output corresponding to the class token is kept and projected into the shared embedding space, where it can be compared with text. The configuration of the CLIP model adopted in this thesis is summarized in Table 1, and the image-encoding pipeline is shown in Figure 2.

Table 1. Configuration of the CLIP ViT-B/32 model.

<i>Component</i>	<i>Value</i>
<i>Pre-trained checkpoint</i>	openai/clip-vit-base-patch32
<i>Image Encoder</i>	Vision Transformer (ViT-B/32)
<i>Input image size</i>	224 x 224 pixels
<i>Patch size</i>	32 x 32 pixels
<i>Number of patches</i>	49 (7 x 7)
<i>Image sequence length</i>	50 tokens (49 patches + class token)
<i>Encoder blocks(per encoder)</i>	12
<i>Text tokenizer</i>	Byte Pair Encoding (BPE)
<i>Text context length</i>	77 tokens
<i>Shared embedding dimension</i>	512
<i>Embedding normalization</i>	L2 (unit length)
<i>Similarity measure</i>	Cosine similarity

The image encoder adopted in this thesis is the ViT-B/32 configuration of CLIP. This variant was adopted over larger configurations such as ViT-L/14 as a deliberate trade-off: with 32×32 patches it produces a shorter sequence and a smaller network, which lowers memory use and speeds up encoding, at a modest cost in accuracy that is acceptable for the dataset size and the interactive demonstration built in this thesis. In this configuration, the input image is divided into patches of 32×32 pixels, which for an image of 224×224 pixels results in 49 patches; together with the class token, the encoder therefore processes a sequence of 50 tokens through twelve successive blocks. The final state of the class token is projected into a 512-dimensional vector, which constitutes the representation of the image in the shared space. After being normalized to unit length, this vector can be compared directly with the vectors produced by the text encoder, using the cosine similarity.

4.5 Contrastive pre-training

The Vision Transformer and the text Transformer each turn their input into a vector, but on their own there is no reason for an image and its description to end up close together. What ties the two encoders into a single shared space is the way that they are trained, through a procedure called contrastive pre-training, which is the method introduced by CLIP [3].

4.5.1 The training objective

Contrastive pre-training works on a large collection of image-text pairs, where each image is associated with a short text that describes it. During training the data is processed in batches of N pairs. The image encoder produces a vector for each of the N images and the text encoder produces a vector for each of the N texts, and the cosine similarity is computed between every image and every text, giving an $N \times N$ table of scores. In this table the N pairs on the diagonal are the correct (matching) pairs, while the remaining $N^2 - N$ combinations are mismatched pairs. The objective is to make the score of every correct pair high and the score of every mismatched pair low, which pulls each image and its own description together in the space and pushes unrelated images and texts apart [3].

4.5.2 The contrastive loss

This objective is expressed as a classification problem solved in both directions at the same time. Looking along one row of the table, the model must pick the correct text for a given image out of the N possibilities; looking along one column, it must pick the correct image for a given text. For each direction, the similarity scores are turned into probabilities with a softmax and compared against the correct pair using cross-entropy, and the two directions are averaged into a single symmetric loss. Before the softmax, the scores are scaled by a learned parameter called the temperature, which controls how sharply the model separates correct pairs from incorrect ones [3]. For a batch of N pairs, the loss in the image-to-text direction is given in equation (6), and the total loss is the average of both directions, as in equation (7):

$$L_{image} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp\left(\frac{S_{ii}}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{S_{ij}}{\tau}\right)} \quad (6)$$

$$L = \frac{1}{2}(L_{image} + L_{text}) \quad (7)$$

In equation (6) N is the number of pairs in batch, s_{ij} is the cosine similarity between image i and text j , and τ is the temperature. In equation (7) L_{image} is the loss in the image-to-text direction, while L_{text} is the symmetric loss in the text-to-image direction, obtained by swapping the roles of images and texts.

The temperature τ is not a fixed constant but a parameter learned together with the network weights, usually stored as a logit scale that multiplies the similarity scores before the softmax [3]. A small temperature makes the resulting distribution sharp and pushes the model to separate the correct pair strongly from the rest, while a larger one softens it. This parameter matters only during training, where it sets how hard matching and mismatching pairs are driven apart. At retrieval time, the images are ranked directly by their raw cosine similarity, and since the temperature is a single positive factor applied equally to every score, it does not change the order of the results and therefore plays no part in the search itself. Minimizing this loss over many batches is what shapes the shared embedding space whose geometry is later exploited at retrieval time. CLIP was adopted as the basis of this thesis for reasons that follow from this training scheme: it is a dual-encoder, so every image can be embedded once in advance and reused, which means it does not need to be retrained for a specific dataset; and pre-trained weights are openly available. For these reasons the training described above is not carried out in this thesis. The CLIP model is used already pre-trained, and contrastive pre-training is presented because it explains why matching images and texts lie close together in the shared space, which is the property the retrieval method relies on.

4.6 Cosine similarity

Once both encoders produce vectors in the same space, the system needs a way to measure how close a text vector is to an image vector. CLIP uses cosine similarity for this purpose. It compares two vectors by the angle between them rather than by their absolute magnitude [3]. For two vectors a and b , it is the dot product divided by the product of their lengths, as given in equation (8):

$$similarity(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \quad (8)$$

Cosine similarity takes values between -1 and 1. A value close to 1 means the two vectors point in nearly the same direction and are considered very similar, a value around 0 means they are unrelated, and a negative value means they point in opposite directions. When the two vectors are normalized to unit length, the denominator becomes 1 and the cosine similarity is equal to the dot product of the vectors. Normalizing the embeddings in this way places them on the unit sphere, so that the comparison depends only on their direction and not on their magnitude.

In practice, the comparison is always done on normalized vectors. Each embedding produced by the two encoders is divided by its own Euclidean norm, so its length becomes one while its direction stays the same. This is an L2 normalization, applied to every image vector and every query vector in this thesis before any score is computed. Once both vectors lie on the unit sphere,

the denominator of the cosine formula equals one, so the cosine similarity reduces to a plain dot product. The dot product is cheaper to evaluate, and it is the form actually used at search time.

Cosine similarity was adopted rather than a magnitude-based distance such as the Euclidean distance, because in CLIP's shared space only the direction of the embeddings is meaningful, which makes an angle-based measure the natural choice [3].

In a text-to-image retrieval setting, the query is encoded into a vector and its cosine similarity with the vector of every image in the collection is computed. The images are then ordered by this score, and the most similar ones (the top-K) are returned as result.

4.6.1 Retrieval

In a text-to-image retrieval setting, the query is encoded into a vector and compared with the image vectors in the collection. Because the two encoders are separate, the image vectors do not depend on the query, so they can be computed once in advance and stored, and at search time only the query passes through the text encoder [3]. Finding the best matches is then a nearest-neighbour search in the shared space. Each image I in the collection receives a score equal to the cosine similarity between its vector v_i and the query vector q , as in equation (9):

$$s_i = q \times v_i \quad (9)$$

In equation (9) q is the query vector and v_i is the vector of the image i , both normalized to unit length.

Since both vectors have unit length, this cosine similarity is simply their dot product. The images are ranked by this score, and the K with the highest values (top-K) are returned as the result.

For a small collection the query can be compared against every image directly, which is what is done in this thesis. When the collection grows to millions of vectors, this exhaustive comparison becomes slow, and specialized indexes for approximate nearest-neighbour search, such as FAISS, are used instead to keep retrieval fast [15]. In both cases the quantity being compared is the same cosine similarity described above, and only the way the closest vectors are found changes.

4.6.2 Re-ranking

The first comparison uses only the visual embedding of each image. It is fast, but its top results can be improved by a second step called re-ranking, which reorders the first list using extra information.

In this thesis the re-ranking uses the human written captions that come with the dataset images. After the first stage returns a short list of candidates by cosine similarity, each candidate is looked at again through its captions. The query is compared, again by cosine similarity in the same shared space [3], with each caption of the candidate image, and the highest of these caption scores becomes the new score of that candidate, as in equation (10):

$$s'_i = \max_j (q \times t_{i,j}) \quad (10)$$

In equation (10) $t_{i,j}$ is the vector of the j -th caption of candidate image I , normalized to unit length, and the maximum is taken over all captions of that image.

The candidates are then reordered by these new scores s'_i , while images with no caption keep their score from the first stage.

This helps because the second comparison is performed between two text embeddings rather than between a text embedding and an image embedding. The captions describe the content of each image in words, so matching the query against them stays within one modality. This is usually more reliable than matching a sentence against a grid of pixels. It can correct some mistakes of the first stage and move the most relevant images higher.

Re-ranking only reorders the candidates chosen in the first stage. It cannot recover a relevant image that the first stage left out of the short list, so its role is to improve the order of the top results, not to widen the set of retrieved images. It was adopted as an optional second stage for this reason: it raises the precision of the top results at almost no cost, since it works on a few candidates instead of the whole collection.

4.7 Text-to-image retrieval pipeline

The previous sections described each component on its own: the two encoders, the shared space they are trained to produce, and the way distances in that space are measured. This section connects them and follows a query through the whole system, from the text typed by the user to the ranked list of images returned at the end.

The vector that an encoder produces for its input is called an embedding. Similar inputs are expected to have nearby embeddings in the shared space. The text encoder turns the query into a text embedding and the image encoder turns each image into an image embedding and both lie in the same 512-dimensional space after being normalized to unit length. Since the two encoders work separately, the image embeddings do not depend on the query, so every image in the collection is encoded once and its embedding is stored ahead of time. At search time only the query has to be encoded, which is what keeps the search fast. The retrieval pipeline therefore consists of two stages. During an offline stage, executed for every used query, only the query is encoded and compared with the stored image embeddings to produce the ranked results. The complete pipeline is shown in Figure 3.

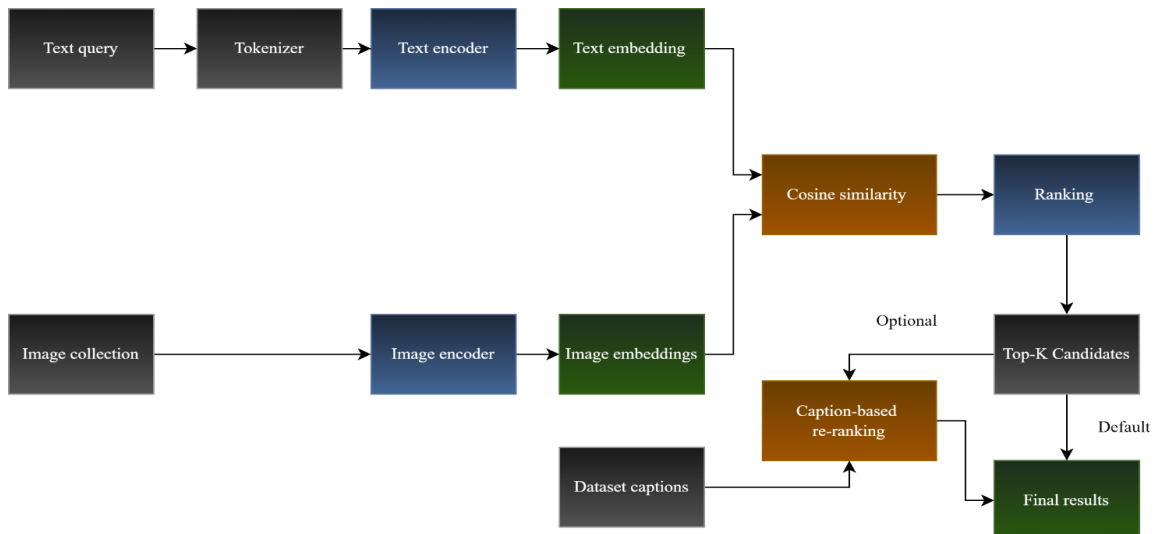


Figure 3. Text-to-image retrieval pipeline.

Similarity search is the step that looks through the stored image embeddings and finds the ones closest to the query embedding. Closeness is measured with the cosine similarity, which for unit vectors is just their dot product, so the query produces one score for every image in the collection. For the dataset sizes used in this thesis, similarity search is performed exhaustively by comparing the query embedding against every stored image embedding.

Ranking orders the images by this score, from the most similar to the least similar. Only the first few matter, so the list is cut to the K images with the highest scores, the top- K and these result of the first stage.

Re-ranking is an optional second stage that changes the order of this short list only. Rather than comparing the query against the images a second time, it compares the query against the human-written captions of each candidate and keeps the best caption score as the candidate's new score. This comparison is performed between text embeddings rather than between text and image embeddings, which is usually more reliable than matching a sentence against an image, so it can move the more relevant images higher up. With or without this second stage, the images left at the top of the list are what the system returns.

4.8 Evaluation metrics

Retrieval systems are commonly evaluated using both quantitative and qualitative methods. Quantitatively means computing a numerical score against answers that are known to be correct. Qualitative methods involve looking directly at the images it returns.

The usual quantitative measure in the image-retrieval literature is Recall at K ($R@K$). When the correct image for each query is known in advance, $R@K$ is the fraction of the queries whose correct image shows up among the top- K results, and it is normally reported for K equal to 1, 5 and 10. $R@1$ is the hardest to satisfy because the correct image has to be in first place, while $R@5$ and $R@10$ allow it anywhere in the first five or ten. The metric only works when every query has a known correct image, as the caption-image pairs of Flickr30k do [16].

The quantitative evaluation in this thesis uses the Flickr30k caption-image pairs. Each caption is treated as a text query, while its associated image is considered the correct retrieval target. For every query, the retrieval system returns the top- K most similar images and the performance is measured using Recall@ K ($R@1$, $R@5$ and $R@10$), which indicates whether the correct image appears among the retrieved results.

The evaluation of the hand-written queries in this thesis is qualitative. Unlike the Flickr30k captions used for quantitative evaluation, these queries do not have a single correct image fixed in advance, so Recall@ K cannot be computed for them. The system is judged instead by looking at its output: for each query, the top- K images are shown together with their similarity scores and how well they fit the query is decided by eye. The same approach is used to compare the two stages of the pipeline by putting the baseline results and the re-ranked results next to each other for the same query and assessing which ranking better matches the query. This qualitative analysis complements the quantitative evaluation by providing representative examples of the system's behaviour.

5. Implementation

5.1 Introduction

The previous chapter described each component of the retrieval system on its own and in the abstract: the two encoders, the shared embedding space they are trained to produce, the cosine similarity that measures the distance in that space, and the optional re-ranking stage. This chapter describes how those components are turned into a working program. The presentation follows a top-down approach: it begins with the overall architecture of the system, shown as a block diagram in Section 5.3 and then descends into each block in turn: the preparation of the data, the encoding of images and queries, the similarity search, the caption-based re-ranking, and the interactive interface that lets a user query the system directly.

The implementation has two parts. The first is an offline stage that prepares and verifies the dataset, computing its statistics and checking its integrity before use; this part is described in Section 5.4. The second is the retrieval system itself, which loads the pre-trained CLIP model, encodes the image collection once into a stored matrix of embeddings, and then answers a text query by comparing it against those embeddings. As justified in Section 4.5.2, no training is carried out. The CLIP model is used with its publicly available pre-trained weights, so the implementation deals only with inference, indexing and retrieval, not with optimizing the network. This separation mirrors the offline and online stages of the theoretical pipeline, where the image embeddings are computed in advance and only the query is processed at search time.

The system is built in Python, using PyTorch [20] for tensor operations and the HuggingFace Transformers library [14] to access the CLIP model. The supporting libraries and the development environment are listed in Section 5.2. The chapter describes the structure of each module, the procedures it carries out and the design choices behind it, while the full commented source code is provided in the Appendix. The aim is to explain how the system is constructed and how a query flows through it. The experimental evaluation, together with the results obtained over collections of increasing size, is presented separately in the following chapter.

5.2 Technologies and development environment

The system is implemented in Python, a language whose ecosystem provides mature libraries for deep learning, numerical computation and data handling, which is why it is the standard choice for this kind of work. All components described in the rest of the chapter are built on a small set of libraries, each selected for a specific role in the pipeline, and they are developed and run inside a single integrated environment. This section presents those libraries, the environment in which the code is written and executed, and the way the chosen technologies come together to load the CLIP model.

5.2.1 Programming language and libraries

The components of the system are built on a small set of Python libraries, each selected for a specific role in the pipeline. They are summarized in Table 2 and described below. The project runs inside an isolated virtual environment so that these library versions are fixed and the results can be reproduced.

Table 2. Software environment and libraries used in this implementation

Component	Version	Role in the system
Libraries		
PyTorch	2.12 (CUDA12.8)	Tensor operations, inference, L2 normalization, GPU acceleration
HuggingFace Transformers	5.3.0	CLIP model and preprocessing (CLIPModel, CLIPProcessor)
Pandas	3.0.1	Reading and cleaning the dataset, caption lookup
Pillow (PIL)	12.2.0	Image loading and RGB conversion
Matplotlib	3.10.8	Result grids and comparison figures
Gradio	6.15.2	Interactive demonstration
Tools		
Python	3.14	Implementation Language
Visual Studio Code	1.125.1	Development environment (code editor / IDE)
Jupyter Notebook	-	Interactive notebook format for organizing and running code

The core of the system is PyTorch [20], the deep-learning framework that handles every tensor operation in the pipeline: encoding images and text, normalizing the resulting vectors and computing the similarity scores. It was chosen for two reasons. The pre-trained CLIP model is distributed for it, and it exposes the low-level operations the implementation depends on, in particular `torch.no_grad()`, which disables gradient tracking during inference to save memory and time, and `F.normalize()`, which performs the L2 normalization. PyTorch also uses the GPU automatically through CUDA when one is available, which is what makes encoding the full image collection practical.

The CLIP model is not implemented by hand but accessed through the HuggingFace Transformers library [14], which provides both the network and the matching preprocessing through two classes. `CLIPModel` loads the pre-trained weights and exposes the image and text encoders separately, and `CLIPProcessor` resizes and normalizes the images and tokenizes the text exactly as the model expects. Using this library rather than a custom implementation guarantees that the preprocessing matches the one used when CLIP was trained, which is essential for the embeddings to be meaningful, and it gives direct access to the pre-trained checkpoint named in Table 1.

The remaining libraries support the stages around the model. Pandas [21], built on the NumPy numerical library [22], is used to read and clean the dataset, which is stored as the pipe-separated `results.csv` file, and to look up the captions associated with each image during re-ranking and evaluation. The Pillow imaging library opens the image files and converts them to the RGB format the model requires. Matplotlib [23] produces the visual output of the system: the grids of retrieved images and the side-by-side comparison figures used throughout the evaluation. Finally, Gradio [24] is used to build the interactive web interface, which lets a user type a query and view the retrieved image directly in the browser, without running any code.

5.2.2 Development environment

The code is written and run inside Visual Studio Code, using its integrated support for Jupyter notebooks, so that the implementation is organized into cells that can be executed and inspected step by step. This notebook format is what allows the intermediate results, such as the encoded embeddings and the retrieved images, to be examined directly during development, which is well

suited to a retrieval system whose output is visual and is best judged by looking at it. The implementation is split across two notebooks: one prepares and verifies the dataset, and the other builds the retrieval system itself, described in the sections that follow.

5.2.3 Model loading and execution device

The choice of execution device is made automatically at start-up: the system queries PyTorch for the availability of a CUDA-capable GPU and selects it when present, falling back to the CPU otherwise. Every model and tensor is then moved to the selected device. Within a single evaluation, image embeddings are computed once and reused for all queries. However, they are not saved between evaluation runs. Therefore, each new evaluation (or each new search session in the Gradio interface) starts by encoding the selected image subset again. The GPU is therefore the most useful during these encoding passes, which dominate the running time, where it reduces the encoding time substantially compared to the CPU.

The way the chosen technologies come together at start-up is shown in Figure 4: PyTorch selects the execution device, and the Transformers library loads the pre-trained CLIP checkpoint together with its preprocessor. The model is loaded directly in evaluation mode, since it is used only for inference and is never trained in this work.

```
# We load a pretrained CLIP model
model_id = "openai/clip-vit-base-patch32" # the specific model we use ViT-B/32 decent
model = CLIPModel.from_pretrained(model_id).to(device) # load the model and move it to the device
processor = CLIPProcessor.from_pretrained(model_id) # the processor handles resizing and normalizing images + tokenizing text
model.eval() # sets model into evaluation mode
print(f"Model loaded: {model_id}")
```

Figure 4. Loading the pre-trained CLIP model.

In Figure 4, the checkpoint loaded here, `openai/clip-vit-base-patch32`, is the ViT-B/32 configuration of CLIP whose parameters are summarized in Table 1. From this point on, the model object exposes the two encoders used throughout the rest of the chapter, `get_image_features()` for images and `get_text_features()` for text, while the processor performs the preprocessing each encoder expects.

5.3 System architecture

The previous section described the technologies the system is built on. This section presents how they are assembled into a complete retrieval system and follows the data through it from end to end.

5.3.1 Overview of the pipeline

The overall architecture is shown in Figure 5. It is organized as the text-to-image retrieval pipeline presented in the previous chapter but drawn here at the level of the actual modules of the implementation, together with the dataset that feeds them and the outputs they produce. The system is built around the two encoders of CLIP, which define two separate processing paths that meet at the similarity-search step: an image path that turns the collection into a set of stored vectors, and a query path that turns the user’s text into a single vector in the same space. The sections that follow take each block of the diagram in turn, in the order a query follows through the system: the dataset

that feeds it (5.4), the image path (5.5), the query path and the search (5.6), the re-ranking stage (5.7), the evaluation methods (5.8), and the interactive interface (5.9).

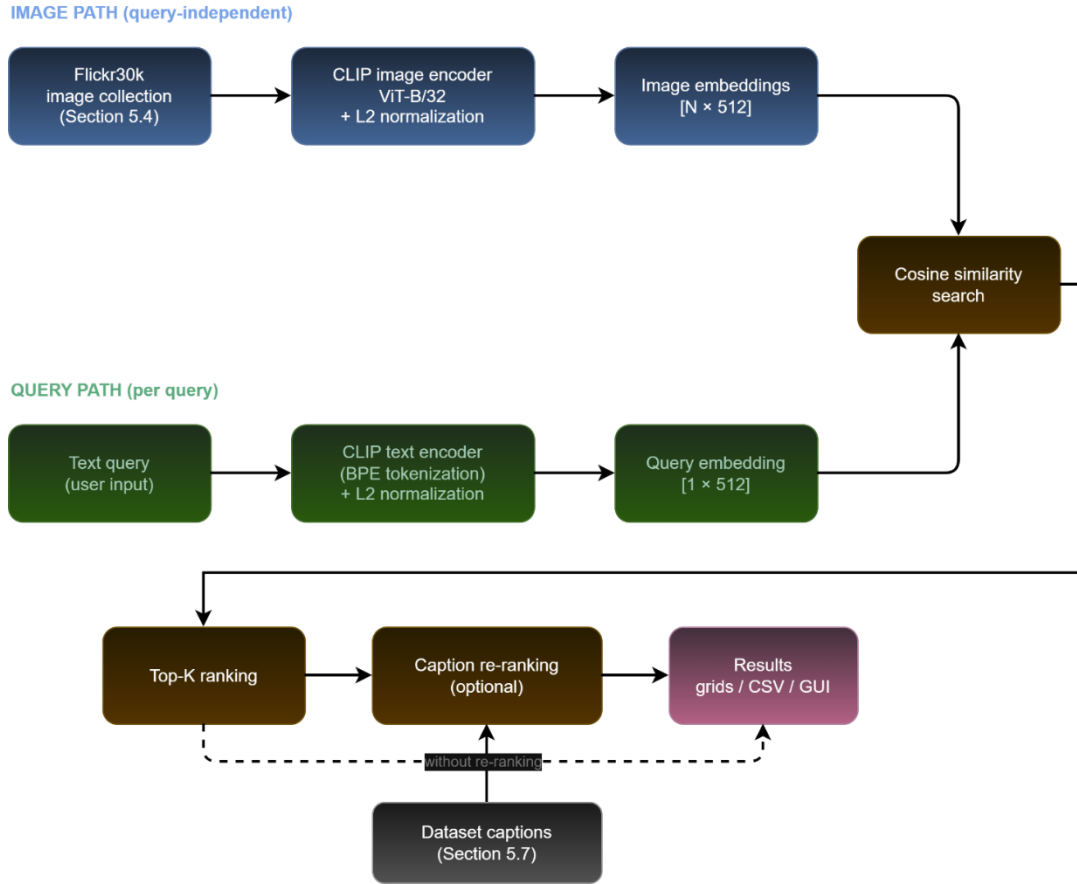


Figure 5. Implementation System Architecture.

5.3.2 Global configuration and parameters

Before the pipeline of Figure 5 can run, a small set of global elements is set up once and shared by every module. The libraries are imported, the execution device is selected, locations of the data and of the saved results are fixed, and two parameters that govern retrieval are set. Figure 6 and Figure 7 show them. The paths point to the Flickr30k images and to the results.csv file of captions, and OUTPUT_DIR is the folder that holds the results figures and CSV files produced during evaluation.

```

import os, csv, torch          # file ops, CSV I/O, PyTorch
import pandas as pd           # dataset handling
import matplotlib.pyplot as plt # result figures
from PIL import Image         # image loading
from transformers import CLIPModel, CLIPProcessor # CLIP + preprocessing
import torch.nn.functional as F # L2 normalization
import gradio as gr          # interactive demo

# execution device: GPU when available, otherwise CPU
device = "cuda" if torch.cuda.is_available() else "cpu"

```

Figure 6. Global configuration: imports, execution device.

```

IMAGE_DIR = r"C:\licenta\dataset\flickr30k_images" # image files
CSV_PATH  = r"C:\licenta\dataset\results.csv"      # captions
OUTPUT_DIR = r"C:\licenta\rezultate_evaluate"      # saved results and figures
os.makedirs(OUTPUT_DIR, exist_ok=True)             # create output folder if missing

TOP_K = 5 # images returned per query
BATCH_SIZE = 32 # images encoded per batch

```

Figure 7. Global configuration.

Two parameters control the behavior of the pipeline. `TOP_K` fixes how many images are returned for each query and is set to 5, the value used consistently throughout the evaluation. In the interactive interface, `K` can be selected by the user. `BATCH_SIZE` fixes how many images are encoded together in a single pass of the image encoder; it is set to 32 as a compromise between speed and memory use, since a larger batch encodes faster but requires more GPU memory, and it can be raised or lowered to match the memory available on the machine. Both values are defined here, in one place, so that they apply uniformly to every stage of the system.

5.4 Dataset preparation

Before any retrieval can be performed, the dataset has to be read, understood and checked. This is the task of the first notebook, which is kept separate from the retrieval system so that the data is prepared and validated once, independently of the model. This section describes the structure of the dataset used, the statistical analysis carried out on it, and the verification and cleaning steps applied before it is handed to the retrieval system.

5.4.1 Structure of the Flickr30k dataset

The dataset used in this work is Flickr30k [16]. It contains 31,783 images of everyday scenes, and each image is described by five short captions written independently by different people, which gives 158,915 captions in total. The images are stored as JPEG files in a single folder. The captions are stored in a text file, `results.csv`, whose fields are separated by the pipe character `|`. Each row holds one caption in three fields: the image file name, the caption number for that image (0 to 4), and the caption text. Table 3 summarizes the dataset.

Table 3. Summary of the Flickr30k dataset.

Property	Value
Images	31,783
Captions	158,915
Captions per image	5
Average caption length	13.39 words
Longest/Shortest caption	82 / 2 words
Image format	JPEG (.jpg)
Storage	results.csv + image folder

Flickr30k was chosen for two reasons that matter to this implementation. First, its size is moderate. With about 31,000 images, the query can be compared against every image directly, without the approximate-search indexes that very large collections need, which keeps the system simple. Second, each image carries five human-written captions, and these are exactly what the re-ranking stage and the Recall@K evaluation rely on, so the dataset supports both stages without any extra annotation.

The dataset is read with pandas, as shown in Figure 8. The pipe character is given as the separator, surrounding whitespace is stripped from the column names so the fields can be accessed reliably, and the list of unique image names is extracted, which confirms the expected 31,783 images.

```
df = pd.read_csv(PATH_CSV, sep="|", on_bad_lines="skip", skipinitialspace=True)
print(f"Unique images: {df['image_name'].nunique()}") # 31,783
```

Figure 8. Reading the dataset and extracting the image list.

5.4.2 Statistical analysis

Before using the data, a short statistical analysis was carried out to understand the nature of the captions. The length of each caption was measured as its number of words, and from this the average and the extreme values were computed, as shown in Figure 9. The average caption is 13.39 words long, the shortest has 2 words and the longest has 82. Figure 10 shows the distribution of caption lengths: most are short, clustered around ten to fifteen words, with a long thin tail of a few much longer descriptions.

```
df["word_count"] = df["comment"].astype(str).apply(lambda x: len(x.split()))
print(f"Average description length: {df['word_count'].mean():.2f} words") # 13.39
print(f"Longest description: {df['word_count'].max()} words") # 82
```

Figure 9. Computing caption-length statistics.

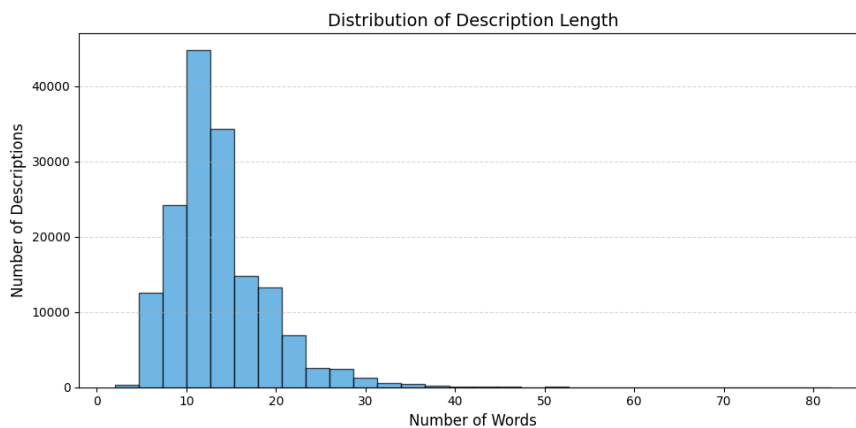


Figure 10. Distribution of caption lengths in Flickr30k.

This confirms that the captions are short, single-sentence descriptions, for two reasons. First, they fit comfortably within the 77-token limit of the CLIP text encoder, so almost no caption is truncated when it is encoded. Second, the queries written for the evaluation can be kept in the same short, descriptive style as the data the model was trained on. As a final qualitative check, the notebook also displays random images alongside their five captions, which makes it possible to confirm by eye that the captions genuinely describe their images before the data is used.

5.4.3 Integrity verification and cleaning

The retrieval system and the evaluation both assume that every image referenced in the captions file exists and that no caption is empty, so the dataset was cleaned and verified before use. An initial inspection found one malformed row at line 19999, where a missing `|` separator left the fields out of alignment and this row was corrected by hand. Duplicate copies of the captions file and the image folder, left over from earlier work, were also removed, so a single unambiguous copy of the data remains. As a further safeguard, the file is read with the `on_bad_lines = "skip"` option, so any remaining malformed row is skipped rather than stopping the whole process.

Two automatic checks were then run, shown in Figure 11. The first counts the missing values in the table and confirms that there are none, so every image name and every caption is present. The second compares the image names referenced in the captions file with the files present in the image folder and confirms that no referenced image is missing from disk. Only when both checks pass is the dataset treated as ready for the retrieval system described in the following sections.

```
print("Missing values:", df.isnull().sum().sum())           # 0 -> no empty captions

csv_imgs      = set(df["image_name"].unique())
folder_imgs   = set(os.listdir(PATH_IMGS))
missing       = csv_imgs - folder_imgs                     # images referenced but absent
print("Missing files:", len(missing))                     # 0 -> all images present
```

Figure 11. Integrity checks: missing values and image availability.

5.5 Image encoding

The image path of Figure 5 is implemented in a single function, `encode_images`, which turns a collection of images into the matrix of embeddings that every query is later compared against. This is the most computationally heavy part of the system, because it runs the image encoder over every image in the subset. Its output does not depend on the query, so within one evaluation it is computed once and reused for all queries. The function does two things in sequence: it loads and prepares the images, then encodes them in batches into normalized vectors.

5.5.1 Loading and preprocessing the images

The encoding function receives a list of image file names and first loads them from disk, as shown in Figure 12. For each name it builds the full path and checks that the file exists, skipping any name with no corresponding file. Each image is opened with Pillow and converted to RGB, because the CLIP image encoder expects three-channel RGB input; an image stored in another mode, such as grayscale, would otherwise cause an error. The loading is wrapped so that an image

which fails to open is reported and skipped rather than stopping the whole process. This keeps the encoding of a large collection robust to a single corrupt file. Alongside the loaded images, the function keeps their paths, so each embedding can later be traced back to the file it came from.

```
def encode_images(image_names):
    images, paths = [], []
    for img_name in image_names:
        img_path = os.path.join(IMAGE_DIR, img_name.strip())
        if not os.path.exists(img_path):
            continue # skip a missing file
        try:
            images.append(Image.open(img_path).convert("RGB")) # CLIP expects RGB
            paths.append(img_path)
        except Exception as e:
            print(f" Error at {img_name}: {e}") # skip an unreadable file
```

Figure 12. Loading the images and converting them to RGB.

The pixels are not resized or normalized here. That is done in the next step by the CLIPProcessor, which resizes each image to 224×224 pixels and normalizes them. Separating the loading from preprocessing keeps the function simple and lets the preprocessing run on a whole batch at once.

5.5.2 Batched encoding and L2 normalization

Once the images are loaded, they are encoded in batches rather than one at a time, as shown in Figure 13. The images are processed in groups of `BATCH_SIZE=32`: encoding several images in a single call to the encoder is far faster than encoding them individually, while a fixed batch size keeps the amount of GPU memory used bounded, regardless of how large the collection is. For each batch, the CLIPProcessor turns the images into the tensor the model expects and moves it to the execution device, and `model.get_image_features()` runs them through the CLIP image encoder, producing one 512-dimensional vector per image (Table 1). For ViT-B/32 `get_image_features()` directly returns the final image embedding used by CLIP.

```
embeddings = []
with torch.no_grad(): # inference only, no gradients
    for start in range(0, len(images), BATCH_SIZE):
        batch = images[start:start + BATCH_SIZE]
        inputs = processor(images=batch, return_tensors="pt",
                           padding=True).to(device) # resize + normalize -> tensors
        output = model.get_image_features(pixel_values=inputs["pixel_values"])
        features = output.pooler_output if hasattr(output, 'pooler_output') else output
        features = F.normalize(features, p=2, dim=-1) # L2 normalize to unit length
        embeddings.append(features.cpu()) # move to CPU to free VRAM

embeddings = torch.cat(embeddings, dim=0) # [N x 512]
return embeddings, paths
```

Figure 13. Batched encoding and L2 normalization of the images.

The whole loop runs inside `torch.no_grad()`, because the model is only used for inference and no gradients are needed. This avoids building the computation graph, which saves both memory

and time. Each batch of vectors is then L2-normalized with `F.normalize()`, so every embedding has unit length and the later cosine-similarity comparison reduces to a dot product. As soon as a batch is produced, its normalized vectors are moved back to the CPU, which frees GPU memory for the next batch, and they are collected in a list. Once every batch is done, the list is concatenated into a single tensor of shape $[N \times 512]$, where N is the number of images successfully loaded. The function returns this embedding matrix together with the list of image paths, the two inputs that `search()` needs.

5.6 Query encoding and retrieval

Once the image embeddings are available, answering a query is the job of the query path in Figure 5. Two functions implement it. The `search()` function encodes the text query and compares it against the stored image embeddings and `evaluate_subset()` drives search over a whole set of queries. Because the two encoders of CLIP are separate, this is the only part of the system that runs for each new query: the image embeddings do not change, so the query is encoded once and compared against them. This section describes how the query is encoded, how the similarity search and ranking are performed, and how the results are displayed and stored.

5.6.1 Query encoding

The `search()` function receives the text query along with the image embeddings and the list of image paths produced by `encode_images()`. The query is first turned into a vector, shown in Figure 14.

```
def search(query, image_embeddings, valid_paths, top_k=TOP_K):
    inputs = processor(text=[query], return_tensors="pt",
                       padding=True).to(device)          # tokenize the query
    with torch.no_grad():
        text_features = model.get_text_features(**inputs)  # CLIP text encoder -> 512-D vector
        text_features = F.normalize(text_features, p=2, dim=-1).cpu() # L2 normalize (unit length)
```

Figure 14. Encoding the text query with the CLIP text encoder.

The `CLIPProcessor` tokenizes the text with the Byte-Pair-Encoding tokenizer and the fixed 77-token context length, and `model.get_text_features()` runs it through the CLIP text encoder, which produces a single 512-dimensional vector representing the whole sentence, handled by the same conditional as on the image side. As with the images, this runs inside `torch.no_grad()`, since only inference is required. The vector is then L2-normalized with `F.normalize()`, exactly as the image embeddings were, so the query and the images lie on the same unit sphere and their scores are directly comparable. Finally, the normalized query vector is moved to the CPU, where the image embeddings are already stored, ready for the comparison in the next step.

5.6.2 Similarity search and top-K ranking

With the query encoded, the search compares it against every image, as shown in Figure 15.

Because both the query vector and the image vectors are normalized to unit length, their cosine similarity reduces to a dot product, so the whole comparison is performed by a single matrix multiplication: multiplying the query vector $[1 \times 512]$ by the transposed embedding matrix $[512 \times$

N] yields a vector of N similarity scores, one for each image, exactly as in equation (9). Computing all scores in one matrix operation is far more efficient than comparing the query against each image separately. The `topk` operation then selects the indices of the `top_k` highest scores, which are the most similar images. For each of these, the function builds a small record holding its rank, its file name, its full path and its similarity score rounded to six decimals and returns the list of records ordered from the most similar to the least similar. These records are the result of the first retrieval stage.

```
similarities = torch.matmul(text_features, image_embeddings.T).squeeze(0) # cosine = dot product
top_indices = similarities.topk(top_k).indices # K highest scores
results = []
for i, idx in enumerate(top_indices):
    results.append({
        "rank": i + 1,
        "image_name": os.path.basename(valid_paths[idx]),
        "image_path": valid_paths[idx],
        "similarity_score": round(similarities[idx].item(), 6),
    })
return results
```

Figure 15. Similarity search and selection of the top-K images.

A single query is rarely evaluated on its own, instead the system is tested on a set of queries at once, which is the role of `evaluate_subset()`, shown in Figure 16. It first encodes the given image subset a single time with `encode_images()` and then runs search for every query against those same embeddings, collecting the results in a dictionary that maps each query to its list of retrieved images. Encoding the subset once and reusing it for all queries makes the evaluation efficient, and it is the direct application of the query-independent image path discussed before.

```
def evaluate_subset(image_names, queries):
    embeddings, paths = encode_images(image_names) # encode the subset once
    if len(paths) == 0:
        print("Error no images found.")
        return {}
    results = {}
    for q in queries:
        results[q] = search(q, embeddings, paths, top_k) # reuse embeddings for every query
        top = results[q][0]
        print(f" '{q}' -> {top['image_name']} (score: {top['similarity_score']:.4f})")
    return results
```

Figure 16. Running the search for every query over a subset of images.

5.6.3 Presentation and storage of results

The dictionary of results produced by `evaluate_subset()` is used in two ways. For visual inspection, the function `plot_results()` displays, for each query, its top-K images side by side in a single row, each labelled with its rank, its similarity score and its file name, which is the form in which the retrieved images are judged by eye. For a permanent record, the function `save_csv()`, shown in Figure 17, writes the results to a comma separated file named after the subset size for

example `rezultate_5000_imagini.csv`. Each row stores the subset size, the query, the rank, the image name, the image path, the similarity score and the value of K, so that every retrieved result is preserved together with the conditions under which it was obtained. These CSV files make the results easier to reproduce and compare and are the basis of the analysis in Chapter 6. A comparison function, `load_results()`, reads such a file back and rebuilds the same query to results dictionary, so that saved results can be compared and inspected later without re-encoding the images. This section describes only how they are produced, while the retrieved images and their scores are examined there.

```
def save_csv(all_results, subset_size, top_k):
    path = os.path.join(OUTPUT_DIR, f"rezultate_{subset_size}_imagini.csv")
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["subset_size", "query", "rank", "image_name",
                        "image_path", "similarity_score", "top_k"])
        for query, results in all_results.items():
            for r in results:
                writer.writerow([subset_size, query, r["rank"], r["image_name"],
                                r["image_path"], r["similarity_score"], top_k])
```

Figure 17. Saving the retrieval results to a CSV file.

5.7 Caption-based re-ranking

The `search()` ranks images using only their visual embeddings. Its ordering can be improved by an optional second stage, re-ranking, which reorders the first-stage short list using the human-written captions of the candidate images: comparing the query against a caption is a comparison between two text embeddings, which can be more reliable than comparing a text embedding with an image embedding, so it can push the more relevant images higher.

The procedure is implemented by the function `rerank_with_captions()`. As shown in Figure 18, it begins by running the ordinary `search()` to obtain the top-K candidates from the first stage, and then encodes the query a single time into an L2-normalized vector. This query vector is compared against the captions of every candidate, so encoding it once and reusing it avoids running the text encoder again for each image.

```
def rerank_with_captions(query, image_embeddings, valid_paths, df):
    initial_results = search(query, image_embeddings, valid_paths)  # first-stage top-K

    q_in = processor(text=[query], return_tensors="pt",
                    padding=True, truncation=True).to(device)  # encode the query once
    with torch.no_grad():
        query_vec = model.get_text_features(**q_in)
        query_vec = F.normalize(query_vec, p=2, dim=-1).cpu()
```

Figure 18. Encoding the query for re-ranking.

Then, for each candidate image, the procedure looks up its captions in the dataset, encodes them in a single batch and normalizes them and computes the cosine similarity between the query and each caption, as shown in Figure 19. The highest of these caption scores becomes the new score of the candidate, as in equation (10). An image with no captions keeps its original visual score, so it never drops from the list. Finally, the candidates are sorted by their new scores and their ranks reassigned, and both the original and re-ranked lists are returned so the two can be compared.

```
reranked = []
for r in initial_results:
    captions = df[df["image_name"].str.strip() == r["image_name"]]["comment"].values
    captions = [str(c).strip() for c in captions]
    if len(captions) == 0:
        reranked.append(r); continue # no captions -> keep score
    c_in = processor(text=captions, return_tensors="pt",
                     padding=True, truncation=True).to(device)
    with torch.no_grad():
        caption_vecs = model.get_text_features(**c_in)
        caption_vecs = F.normalize(caption_vecs, p=2, dim=-1).cpu()
    max_score = torch.matmul(query_vec, caption_vecs.T).max().item() # best caption match
    reranked.append(**r, "similarity_score": round(max_score, 6))

reranked.sort(key=lambda x: x["similarity_score"], reverse=True) # reorder by new score
for i, r in enumerate(reranked):
    r["rank"] = i + 1
```

Figure 19. Re-scoring each candidate by its best caption and re-ordering.

A reporting function, `show_rerank_changes()`, reports how much the ranking changed by printing, for each image, its position before and after re-ranking and the direction it moved. The two scores are not on the same scale: the first-stage score measures query-to-image similarity, while the re-ranked score measures query-to-caption similarity, so the system deliberately does not compare them numerically and reports only the change in position. Re-ranking is applied to several queries to make its effect representative, but the resulting orderings and the comparison between the two stages are presented in Chapter 6.

5.8 Evaluation methods

The system is evaluated in two complementary ways. Qualitatively, it is queried with short descriptive sentences written by hand, and the images it returns are judged by eye, since these queries have no single correct answer fixed in advance. Quantitatively, it is measured with `Recall@K` over the captions of the dataset, which do have a known correct image and therefore allow a numerical score. This section describes both methods and the way the tests are organized; the results they produce are presented in Chapter 6.

5.8.1 Functions for the qualitative evaluation

The qualitative evaluation is built on the `evaluate_subset()` function, which runs a set of queries over a chosen collection and returns, for each query, its ranked list of retrieved images. Around it, three functions support the analysis of these results.

The function `compare_results()` takes the results obtained on two different collections and for each query measures how stable the retrieval is between them. It computes the overlap, that is how many of the top-K images are returned by both collections and the average similarity score and writes these into a CSV file. Comparing two collections in this way shows whether the top results settle as more images are added. The remaining two are helpers for inspection: `plot_comparison` places two result sets side by side and highlights the images they share, and `verify_captions` cross-references each retrieved image with its own captions from the dataset, so that whether an image genuinely matches a query can be checked against the words that describe it. These functions produce only comparative and visual output, which is presented in Chapter 6.

5.8.2 Quantitative evaluation with Recall@K

The quantitative evaluation measures Recall@K ($R@1$, $R@5$ and $R@10$), the standard text-to-image retrieval metric of the literature. It relies on the fact that each caption in the dataset is written for one specific image, so a caption can be used as a query whose correct answer, its own image, is known in advance. This is what makes a numerical score possible, which the hand-written queries do not allow.

Recall@K is reported only for the baseline retrieval, not for re-ranking. Each query is itself a dataset caption, so re-ranking would compare it against the correct image's own captions, which includes that query. The match would be trivial, inflating the score through information leakage.

The method is implemented by `evaluate_recall()`. It first encodes the image collection once and records, for each image, its position in the embedding matrix. It then builds a list of caption queries paired with the index of the image each caption describes, keeping only the captions whose image is present in the collection. The captions are encoded in batches, and for each caption the cosine similarity against all image embeddings is computed and the indices of the highest-scoring images are taken. For every value of K, the caption counts as a hit if the index of its correct image appears among the top-K retrieved images, and the recall is the number of hits divided by the number of caption queries, as shown in Figure 20.

```
max_k = max(k_values)
hits = {k: 0 for k in k_values}
with torch.no_grad():
    for start in range(0, len(captions), text_batch):
        batch = captions[start:start + text_batch]
        inputs = processor(text=batch, return_tensors="pt",
                           padding=True, truncation=True).to(device)
        output = model.get_text_features(**inputs)
        tf = output.pooler_output if hasattr(output, 'pooler_output') else output
        tf = F.normalize(tf, p=2, dim=-1).cpu()
        sims = torch.matmul(tf, embeddings.T)           # [B, N] scores
        topk = sims.topk(max_k, dim=1).indices          # top image indices per caption
        for row, gi in enumerate(gt_idx[start:start + text_batch]):
            ranked = topk[row].tolist()
            for k in k_values:
                if gi in ranked[:k]:                   # correct image inside top-K?
                    hits[k] += 1
recall = {k: hits[k] / len(captions) for k in k_values} # R@K = hits / total
```

Figure 20. Computing Recall@K by counting how often a caption retrieves its own image.

5.9 Interactive demonstration

Beyond the evaluation scripts, the system is made usable interactively through a graphical interface built with Gradio [24], which lets a user type a query and see the retrieved images directly in a web browser, without running any code. The interface is driven by the function `gradio_search`, shown in Figure 21, which ties together the components described in the previous sections.

```
def gradio_search(query, subset_size, top_k):
    image_names = all_images[:int(subset_size)]      # slice the dataset to the chosen size
    embeddings, paths = encode_images(image_names)    # encode on the fly (fine for a demo)

    results = search(query, embeddings, paths)[:top_k] # run search, keep top-K

    images = []
    captions = []
    for r in results:
        try:
            img = Image.open(r["image_path"]).convert("RGB")
            images.append(img)
            captions.append(f"#{r['rank']} scor: {r['similarity_score']:.3f} | {r['image_name']}")
        except Exception:
            pass # skip an image that fails to load
    return list(zip(images, captions))                # Gradio Gallery expects (image, caption) tuples
```

Figure 21. The search function behind the interface.

The function receives the query text together with two parameters chosen by the user: the number of images to search over and the number of results to display. It slices the dataset to the chosen size, encodes those images with `encode_images()`, runs `search()` and keeps the top-K results, and finally loads each retrieved image and pairs it with a short caption showing its rank, similarity score and file name, in the form the gallery (a list of images-caption) expects. The interface itself is defined with `gr.Interface`, shown in Figure 22. It exposes a text box for the query, a slider for the subset size, ranging from 100 to all 31,783 images with a default of 5,000 images and a slider for the number of results K, from 1 to 5, while the retrieved images are shown in a gallery.

```
demo = gr.Interface(
    fn=gradio_search,
    inputs=[
        gr.Textbox(label="Query text"),
        gr.Slider(100, 31783, value=5000, step=100, label="Subset imagini"),
        gr.Slider(1, 5, value=5, step=1, label="Top-K rezultate"),
    ],
    outputs=gr.Gallery(label="Results"),
    title="CLIP Image Search",
    description="Search images in Flickr30k using a text query."
)
demo.launch()
```

Figure 22. Building and launching the Gradio interface.

6. Experimental Results

6.1 Experimental Setup

The previous chapter described how the retrieval system was built. This chapter presents the results of running it. No new code is introduced here, every result comes from the search and evaluation methods of the previous chapter, applied under the conditions fixed in this section. The chapter first presents the retrieved images and how their quality is judged by eye (6.2), then examines how the results change as the image collection grows (6.3), verifies the retrieved images against their original captions (6.4), evaluates the caption-based re-ranking stage (6.5) and measures Recall@K on caption queries (6.6). Section 6.7 demonstrates the interactive interface and Section 6.8 discusses what the results show about the system as a whole.

All experiments run on a single laptop with an AMD Ryzen 7 7435HS processor (8 cores, 16 threads), 16 GB of RAM, and an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB. The device selection step detects this GPU through CUDA, so all image and text encoding runs on it. Encoding the image collection dominates the running time, which is why the subset sizes used below are practical. The software environment is the one listed in Table 2 and the retrieval parameters keep the values fixed in the global configuration: every query returns the TOP_K = 5 best images, and images are encoded in batches of 32.

The evaluation uses the eight text queries listed in Table 4. They are written by hand for this work rather than taken from the Flickr30k captions, for two reasons. First, this mirrors how the system is meant to be used, where a user describes the desired image in their own words, and it guarantees that no query can succeed by trivially matching a caption already stored in the dataset. Second, the queries keep the same short, descriptive style as the captions the model is compared against, since such sentences fit well within the 77-token limit of the text encoder. The eight queries are chosen to cover different kinds of scenes: people in groups and alone, animals, objects, urban scenes, and a landscape, so the system is tested across varied subjects rather than a single theme. These hand-written queries are used for qualitative evaluation

Table 4. The eight test queries used throughout the evaluation.

#	Query
1	a dog playing in the park while a woman watches nearby
2	people walking on a city street with umbrellas while laughing
3	a child holding a red balloon in a field of flowers
4	sunset over a mountain range with a lake in the foreground
5	a group of friends having a picnic in the park on a sunny day
6	a woman sitting at a cafe table with a laptop and a cup of coffee
7	cars on a busy street with tall buildings in the background
8	group of people playing soccer on a grassy field with trees around it

Each query is run against the five image subsets listed in Table 5, from 500 images up to the full collection of 31,783. A subset of size N consists of the first N entries of the image list, in the order they appear in the captions file, so the subsets are nested: every image in a smaller subset is also in all the larger ones. This nesting is what makes the comparison across the growing subsets meaningful, because when the collection grows, a result can only leave the top 5 because a newly added image scored higher, not because the image itself disappeared from the collection or was skipped.

Table 5. Image subsets used in the evaluation.

Subset	Number of images	Share of the collection
500 img	500	1.6 %
5000 img	5,000	15.7 %
10000 img	10,000	31.5 %
20000 img	20,000	62.9 %
All images	31,783	100 %

The hand-written queries have no single correct image fixed in advance, so the experiments are judged qualitatively by inspecting the returned images and their similarity scores. The quantitative measurement instead uses the Flickr30k captions themselves as queries, for which the correct image is known, so Recall@K can be computed.

6.2 Qualitative retrieval results

The first experiment examines the quality of the retrieved images directly. Each of the eight queries of Table 4 is run against the full collection of 31,783 images and the top-5 results are displayed with `plot_results()`, each image labelled with its rank, its similarity score and its file name. Three representative queries are discussed here: a typical query that retrieves what it asks for, the best scoring query of the set, and the most difficult one.

Figure 23 shows the results for the query “a dog playing in the park while a woman watches nearby”. Three of the five retrieved images match the query well: dogs on grass in park settings although only one of them includes a woman watching nearby, with similarity scores between 0.332 and 0.339. The other two results are particularly informative because they reveal different types of retrieval errors: the image ranked first, with the highest score of 0.350, shows a park with a bench and a child sitting on it and the image ranked third shows a fenced city park with one person talking to another, but neither contains a dog. Both match the setting of the query rather than its subject, which illustrates a known limitation of the dual-encoder design: the whole image and the whole sentence are each compressed into a single vector, so a strong match on the overall scene can outweigh a missing object. The encoder rewards overall semantic agreement and cannot enforce the presence of one specific object.



Figure 23. Top-5 results for query 1 on the full collection.

Figure 24 shows the best-scoring query of the set, “a woman sitting at a cafe table with a laptop and a cup of coffee”, whose first result reaches 0.361, the highest similarity observed across all eight queries. The first-ranked image satisfies every element of the sentence at once: a woman

seated at a cafe table, a laptop in front of her and a drink beside it. The second image still contains a woman and a table and a laptop, in a darker interior. The third shows a woman at a cafe table with a large coffee cup reading a book but no laptop and the last two show women at cafe tables, one outside and one inside, but only with drinks. The ranking therefore follows the number of query elements each image satisfies: the full description first (0.361 and 0.351), the image missing the laptop next (0.342), and the images matching only “a woman sitting at a cafe table” last (0.332 and 0.322). This gradual decline of the score as query elements disappear is the complement of the observation made on Figure 23: the shared embedding space rewards each matched concept, which works in the query’s favor when the collection actually contains an image satisfying all of them.



Figure 24. Top-5 results for query 6 on the full collection.

Figure 25 shows the most difficult query, “sunset over a mountain range with a lake in the foreground”. The scores are visibly lower than for the other queries, from 0.271 down to 0.259 against roughly 0.32 to 0.36 elsewhere, and the retrieved images show why: each of them satisfies only part of the description. The first two combine a sunset, mountains and water, the third and fourth are sunsets over water with no mountains in sight, the fifth is a mountain lake photographed in daytime, with no sunset. A person or a boat also appears in every one of them. The reason lies in the collection rather than the model. Flickr30k consists of everyday scenes centered on people and their activities and contains almost no pure landscape photographs, so the best available matches are images in which the landscape shares the frame with human activity. The system still places these best-available images at the top, which is the correct behaviour for a retrieval system.

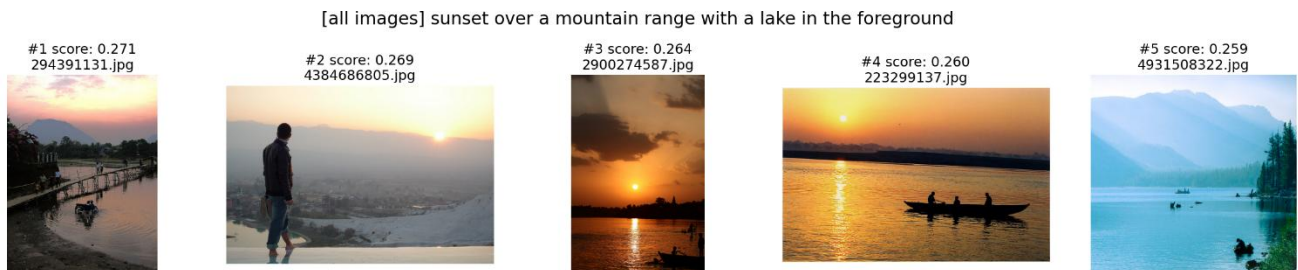


Figure 25. Top-5 results for query 4 on the full collection.

Taken together, the eight queries place their top results between 0.271 and 0.361 and the differences inside this band reflect how well the collection can serve each query: the score rises towards its upper end when an image satisfies every element of the sentence (Figure 24) and stays low when every candidate misses some of them (Figure 25). A low score can therefore indicate that the collection does not contain a close match, rather than necessarily indicating a failure of the search. The modest level of the band itself has a different cause: every score in this first stage is a cosine similarity between a text embedding and an image embedding, and the contrastive training

only teaches the model to rank a matching pair above the mismatched ones, not to drive their similarity toward 1. Cross-modal scores therefore remain in a compressed range and it is the ordering, not the absolute value, that carries the information. This becomes directly visible in the re-ranking results, where the same query compared against text, the captions of the retrieved images, produces similarities more than twice as high.

6.3 Effect of collection size

The second experiment examines how the retrieval results change as the collection grows. Each query is evaluated on the five nested subsets of Table 5, and consecutive subsets are compared with `compare_results()`, which measures two quantities per query: the average similarity score of the top-5 results and the overlap, that is, how many of the top-5 images are the same in both subsets.

Table 6 shows the average top-5 score for every query and subset size. Because the subsets are nested, the average top-5 score cannot decrease when the collection grows: all previous candidates remain available, and newly added images can only enter the top-5 if they score higher than one of the previous results. The table confirms this for every query. The informative part is the rate of this growth. The average over the eight queries rises by +0.033467 in the first step, from 500 to 5,000 images, but each following step adds less: +0.012534 from 5,000 to 10,000, +0.004623 from 10,000 to 20,000 and only +0.004317 in the final step from 20,000 to the full collection. The gains diminish because each enlargement competes with an already better-served query: once the collection contains a good match, a new image improves the top-5 only if it scores higher than the images already there and the better the current results are the fewer images can beat them. Early enlargements therefore improve the results substantially, while later ones find little left to improve.

Table 6. Average similarity score of the top-5 results for each query, as the collection grows (queries numbered as in Table 4).

Query	500	5,000	10,000	20,000	31,783
1	0.293855	0.314778	0.329383	0.333032	0.337952
2	0.271145	0.314967	0.31976	0.31976	0.324731
3	0.262958	0.301514	0.31693	0.324407	0.326958
4	0.217237	0.252761	0.260154	0.26365	0.264653
5	0.271288	0.298953	0.315424	0.317857	0.320209
6	0.266756	0.31216	0.32063	0.332785	0.341728
7	0.268122	0.28963	0.304753	0.309643	0.315367
8	0.265978	0.300309	0.318312	0.321196	0.325263
Average	0.264667	0.298134	0.310668	0.315291	0.319608
Δ vs previous		+0.033467	+0.012534	+0.004623	+0.004317

Table 7 shows the overlap between the top-5 lists of consecutive subsets. At the small end the lists are replaced almost completely: between 500 and 5,000 images only 2 of the 40 top-5 places survive and between 5,000 and 10,000 only 12. From 10,000 to 20,000 the picture reverses and 28 of the 40 places persist. For query 2 the top-5 is identical in both subsets, images and scores alike, which is why its average in Table 6 does not move between these two sizes. In the final step to the full collection 25 of the 40 places persist, so even the last 11,783 images still displace one or two results per query, but Table 6 shows that these newcomers improve the average by only 0.004317, the results keep changing slightly, while their quality is essentially settled.

Table 7. Overlap between the top-5 lists of consecutive subsets (out of 5).

Query	500-> 5,000	5,000-> 10,000	10,000 ->20,000	20,000 -> 31,783
1	0	0	4	4
2	0	3	5	2
3	0	1	2	4
4	0	3	4	4
5	0	1	4	2
6	0	2	2	3
7	2	2	3	3
8	0	0	4	3
Total	2/40	12/40	28/40	25/40

Figures 26 and 27 show the two ends of this evolution on query 1. In Figure 26, between 500 and 5,000 images, the two rows share no image at all: every result found in the small collection is displaced by a better match. In Figure 27, between the 20,000 image subset and the full collection, four of the five images persist, marked by the green borders and the one newcomer is the bench image discussed in Figure 23, which enters the ranking, directly at first place, where all the images are accounted for.

The two tables together describe one process seen from two sides. While the collection is small, it undersupplies every query, so each enlargement brings strictly better matches and overturns the ranking. As the collection grows, the best matches become representative of what the dataset can offer, the scores approach a plateau and the top of the ranking stabilizes. For the eight test queries this point is reached at roughly 20,000 images. Beyond it, adding the rest of the collection leaves the quality of the results practically unchanged. This stability also matters for the next sections: the results examined there are the settled ones, not an artifact of a particular collection size.



Figure 26. Comparison of the top-5 results for query 1 between 500 and 5,000 image subsets.



Figure 27. Comparison of the top-5 results for query 1 between 20,000 image subset and the full collection.

6.4 Caption verification

The earlier judgments were made by eye. This section checks them against an independent source: the five human-written captions that accompany every Flickr30k image. The captions were written by annotators who described each image without knowing any query, so if a retrieved image genuinely matches a query, its captions should retell the content of the query in different words. The check is performed by `verify_captions()`, which looks up the captions of the top five results of a query.

Description of the returned images (all images):

Query: 'a dog playing in the park while a woman watches nearby'

#1 449377293.jpg (score: 0.350383)

- > A little girl sits alone on a park bench with city buildings in the background .
- > A little girl about 2 with blond-hair is sitting on a bench all by herself .
- > A little girl in a pink shirt is sitting on a park bench in fall .
- > A little girl sitting on a bench with a ball under it .
- > A little girl is sitting on a bench in a park .

#2 3540416139.jpg (score: 0.339486)

- > A golden dog is running across the grass chasing a white toy on the ground .
- > A dog chases a toy in the grass with its owner in the background .
- > The brown dog is running outside on the grass .
- > A dog chases a white animal on a green lawn .
- > A dog playing outside in the grass .

#3 275290256.jpg (score: 0.334933)

- > Two people hang outdoors along side netted fence in a suburban area .
- > This is what looks to be a ghetto apartment complex .
- > A man dressed in black walks across a bridge .
- > People standing outside of a building .
- > How do we get to the other side ?

Figure 28. Captions of the top-5 retrieved images for query 1.

For the dog query, the captions confirm the visual assessment of Figure 23 exactly. The second-ranked image is captioned “A dog chases a toy in the grass with its owner in the background .” and “A dog playing outside in the grass .”, the annotators describe the scene almost in the words of the query itself. The first-ranked image, however it is captioned only as a little girl sitting on a park bench, one caption noting “a ball under it”, and the third-ranked image as people standing along a netted fence, no caption of either image mentions a dog. What was judged by eye is therefore confirmed by the dataset’s own ground truth: the two suspect results match the setting of the query, not its subject. Figure 28 shows the output for query 1 on the full collection.

For the cafe query, the captions confirm the first result, “a woman with blond-hair is sitting in a booth with a drink working on her laptop.” and the third described as a woman reading and drinking coffee at a coffee shop. They also correct an impression: the second ranked image, which visually passed as a woman with a laptop and a cup, is captioned by the annotators as “a blond woman sitting at a desk in a dimly lit room” and “a blond woman works in a dark office”, an office scene rather than a cafe, so this match is looser than it appeared. Figure 29 shows the output for query 6 on the full collection.

```
Query: 'a woman sitting at a cafe table with a laptop and a cup of coffee'
#1 4983587808.jpg (score: 0.360511)
-> A woman with blond-hair is sitting in a booth with a drink working on her laptop .
-> As I slave over this assignment , I cautiously click on the answer !
-> A woman working on her computer in front of a bright yellow wall .
-> Woman sitting at a table while working on her laptop computer .
-> A woman in a white shirt working on her laptop .
#2 31916682.jpg (score: 0.351121)
-> A blond woman sitting at a desk in a dimly lit room with ladders and metal frame chairs behind her .
-> A blond-haired lady is sitting at a desk with a yellow map hanging on the wall .
-> A blond woman alone in a dimly lit room .
-> A blond woman works in a dark office .
-> A blond woman sits behind a desk .
#3 1547883892.jpg (score: 0.342463)
-> A seated young woman in a hat is reading at a coffee shop .
-> A girl is reading while drinking Starbuck 's coffee .
-> A woman in a hat sits reading and drinking a coffee .
-> A young woman sips a coffee and reads a book .
-> a woman enjoying her reading at a coffee shop
```

Figure 29. Captions of the top-5 retrieved image for query 6.

The verification also exposes a property of the captions themselves: one caption of the first image reads “As I slave over this assignment , I cautiously click on the answer !”, a first person remark that describes nothing in the image. Such noise is a known trait of crowdsourced annotation and it matters for the next section, where these same captions become the text for re-ranking procedure.

In summary, the caption check strengthens the qualitative evaluation, the matches judged correct by eye are described by annotators in terms that repeat the query and the matches judged wrong are described in terms that contradict it. It also grades the results more finely than eye alone, separating genuine matches from ones that only look right.

6.5 Caption-based re-ranking results

The last qualitative experiment evaluates the caption-based re-ranking, applied to queries 1, 3 and 6 of Table 4 on the 5,000 image subset. The first stage retrieves the top-5 images as before, the second rescores each candidate by the best similarity between the query and the candidates captions, as in equation (10) and reorders the list. Because it only reorders, the same five images appear before and after in every case, what changes, as reported by `show_rank_changes()` in Table 8, is their position. The two score columns of the table must not be compared with each other because they are not on the same scale: the image score is a text-to-image similarity and the caption score is a text-to-text similarity, which, as already seen, lives on a visibly higher scale (0.62-0.86 against 0.29-0.35). The meaningful output is the change in position.

Table 8. Rank movements produced by caption re-ranking on the 5,000 image subset.

Query	Image	Rank Before	Rank After	Movement	Score (image)	Score (caption)
1	1262454669.jpg	3	1	Up 2	0.3152	0.8550
1	1289352548.jpg	4	2	Up 2	0.3126	0.8080
1	2274602044.jpg	2	3	Down 1	0.3161	0.7888
1	150521539.jpg	5	4	Up 1	0.3125	0.7610
1	129981966.jpg	1	5	Down 4	0.3175	0.7267
3	181989740.jpg	2	1	Up 1	0.3028	0.7299
3	1386964743.jpg	5	2	Up 3	0.2967	0.6821
3	2312747482.jpg	1	3	Down 2	0.3122	0.6399
3	2053441349.jpg	4	4	Same	0.2970	0.6236
3	174466741.jpg	3	5	Down 2	0.2988	0.6196
6	1547883892.jpg	1	1	Same	0.3425	0.8019
6	132991765.jpg	2	2	Same	0.3122	0.7933
6	2062425492.jpg	5	3	Up 2	0.2986	0.6990
6	18835207.jpg	3	4	Down 1	0.3061	0.6895
6	1388373425.jpg	4	5	Down 1	0.3015	0.6481

For query 1, shown in Figure 30, the correction is decisive.

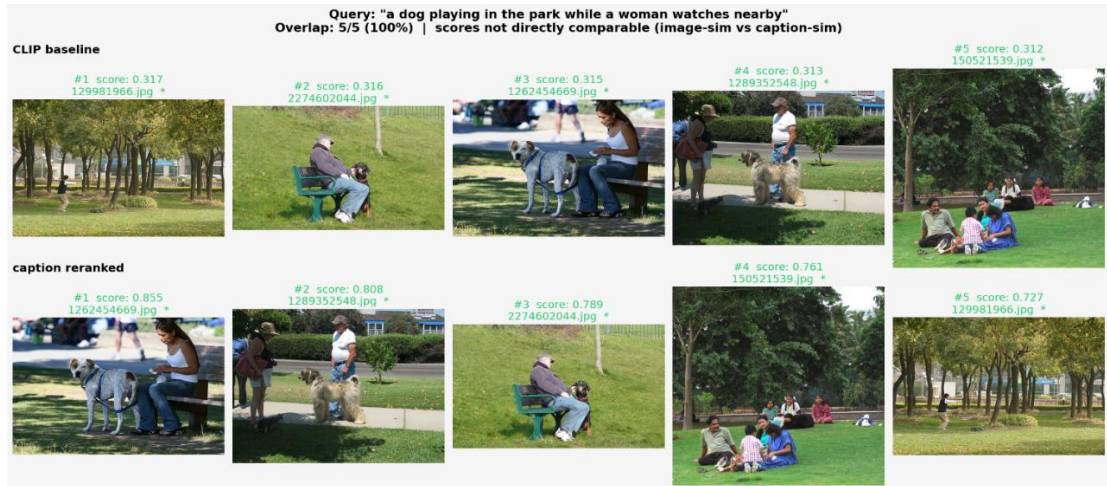


Figure 30. Re-ranked top-5 for query 1 on the 5,000 image subset.

On this subset the first stage makes the same kind of error seen on the full collection in Figure 23, its top ranked image is a park scene without a dog, captioned by annotators as “A man is standing in a forest practicing a martial art .”, and re-ranking sends it to last place, the image promoted to first is captioned “a woman is sitting on a bench with a latte in her lap and a white dog on a blue leash to her side .”, nearly the query itself. Query 3 behaves the same way, the first stage top image contains, by its captions, a child and flowers but no balloon, and it falls to third, while an image captioned “A child holding a helium filled balloon” rises to first. Query 6 shows the complementary case, its baseline top-2 already match the query well, their captions describing women with coffee at tables, and re-ranking keeps both in place, reshuffling only the lower half of the list.

The three queries thus illustrate the behaviour expected of a two-stage system, the fast visual stage occasionally ranks a scene-level match above the true subject and the caption stage repairs exactly those cases, leaves correct results alone and costs almost nothing, encoding only the 25 captions of the five candidates instead of the whole collection.

6.6 Quantitative evaluation: Recall@K

The quantitative measurement uses `evaluate_recall()` on three nested collections of 1,000, 5,000 and 20,000 images. For each collection, all captions of its images serve as queries, five per image, and a caption counts as a hit at K if its own image appears among the top-K retrieved. Table 9 shows the results.

Table 9. Recall@K for text-to-image retrieval on three collection sizes, every caption of the collection’s images serves as a query.

Collection size	Caption queries	R@1	R@5	R@10
1,000	5,000	59.16%	84.50%	90.66%
5,000	25,000	40.33%	66.66%	76.20%
20,000	100,000	25.36%	47.17%	56.85%

On the smallest collection the correct image is ranked first 59.16% of the 5,000 caption queries and appears in the top ten for 90.66% of them, on the largest R@1 falls to 25.36% while R@10 remains at 56.85%.

Two comparisons help in judging these values. The first is against random guessing: with 1,000 images in the subset, a random ranking would place the correct image first in 0.1% of cases, and with 20,000 images in 0.005% of cases, while the system reaches 59.16% and 25.36%. A direct comparison with the Recall@K values published for CLIP on Flickr30k should be avoided. Those results are measured on the standard test split of 1,000 specific images, while the subsets here are simply the first 1,000, 5,000 and 20,000 images of the collection and Table 9 itself shows how strongly recall depends on how many images are searched. The values in Table 9 are therefore internal measurements, they can serve as a baseline if a component of the system, for example the encoder, is ever replaced.

6.7 Interactive demonstration

The system is exercised through a graphical interface in Figure 31. The figure shows the system answering the query “a man riding a bicycle on a mountain trail”, typed directly into the browser, over the default subset of 5,000 images. This query is deliberately not one of the eight test queries of Table 4, then it returns each image labelled with its rank, score and file name exactly the evaluation figure. This shows the system behaving for an unseen query the same way it behaved throughout this chapter.

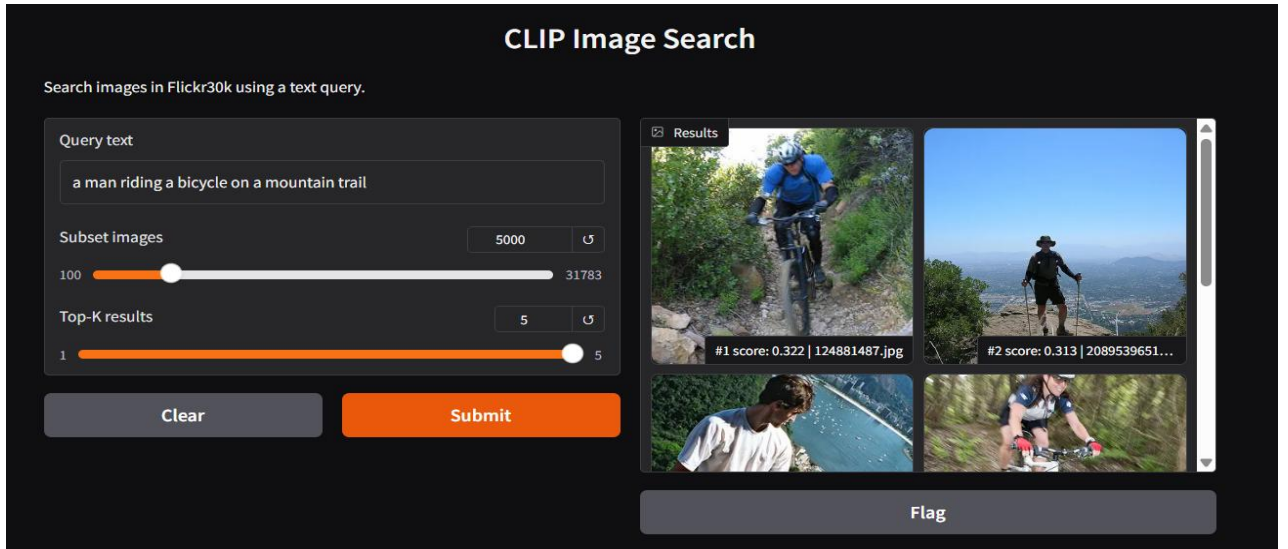


Figure 31. Demonstration of the implementation in a GUI.

The response time is dominated by encoding the selected subset, which the interface redoes for every query. On the test machine, a query over 5,000 images takes about 32 seconds. The retrieval itself, one text encoding and one matrix multiplication, is a negligible part of that time. This is acceptable for a demonstration. The obvious improvement, computing the image embeddings once and storing them, is taken up in the concluding chapter.

6.8 Discussion of results

The results of this chapter fit one picture. The absolute similarity scores of the first stage are low, between 0.26 and 0.36 for the test queries, and this level should not be read as poor retrieval. It follows from what is measured: cross-modal scores are compressed by the contrastive training objective and the pre-trained ViT-B/32 encoder is a general-purpose model that was not fine-tuned on Flickr30k, so its representation of a concept such as “park” must cover far more uses of the word than the everyday outdoor scenes of this dataset. The scores are primarily used for ranking and the experiments indicate that this ordering is generally reliable. The qualitative results place plausible matches at the top even for queries the collection serves poorly, the ranking stabilizes as the collection grows and the dataset’s own captions confirm the interpretation of the retrieved matches.

The measurements also show where the quality ceiling lies and what would move it. The failures observed are of a single kind, a global embedding that matches the scene while missing the subject and the caption-based second stage already repairs these cases at negligible cost, confirming

the value of the two-stage design. Beyond that, the main lever is the image encoder itself. A larger CLIP configuration such as ViT-L/14 processes the image in 14x14 pixel patches instead of 32x32, giving a longer input sequence and a larger network [3] and would be expected to improve both the similarity scores and the recall values of Table 9. Its price is encoding time, which dominates the running time of the system. For a collection of this size and an interactive demonstration, the ViT-B/32 tradeoff remains appropriate and the measurement protocol provides the baseline against which such an upgrade could be measured.

7. Conclusions

7.1 Summary of the work

This thesis addresses multimodal search in the form of image retrieval based on textual descriptions: the user describes an image in a short sentence and the system returns, from a collection of images carrying no manual tags, the images that best match the description. The system built for this purpose follows the retrieve-then-rerank design in Chapter 3. Its first stage is the pre-trained CLIP ViT-B/32 dual encoder [3], which maps the query and the 31,783 images of Flickr30k [16] into a shared 512-dimensional embedding space and ranks the images by cosine similarity. Its optional second stage reorders the top-K candidates by comparing the query against five human-written captions of each candidate, keeping the best caption score. No training is carried out: the model is used with its publicly available pre-trained weights, so the whole system runs as inference on a single laptop and it is exposed through a graphical interface in which a user can type arbitrary queries.

The experiments carried out in the last chapter lead to four main findings. First, the absolute similarity scores of the first stage are modest, the top result of each hand-written query lying between 0.271 and 0.361, but this level reflects how cross-modal scores are compressed by contrastive training, not poor retrieval. The information is carried by the ordering and every experiment that tested the ordering found it reliable. Second, the only systematic failure of the first stage is a global embedding that matches the scene of a query while missing its subject and the caption-based second stage repairs exactly these cases at negligible cost, demoting the false scene-level matches while leaving already correct results in place. Third, the results settle as the collection grows: the average top-5 score rises from 0.264667 on 500 images to 0.319608 on the full collection, with diminishing gains while both the scores and the composition of the top-5 lists are essentially stable by roughly 20,000 images. Fourth, the quantitative measurement confirms the ranking numerically while showing how strongly recall depends on the size of the search set. With captions as queries, R@1 is 59.16% on 1,000 images and falls to 25.36% on 20,000 images, since every added image is a further distractor. These values are internal baselines and are not comparable with the published Flickr30k benchmarks, which use a fixed test split.

7.2 Limitations and future work

The system has two limitations that point directly to its most natural improvements. The first is practical, the image embeddings are not stored between runs, so the interactive interface re-encodes the selected subset for every query, which is where almost all of the roughly 32 seconds per query over 5,000 images is spent. The natural improvement is to compute the image embeddings once and store them, on disk or in memory, so that later queries load them instead of recomputing them. The response time then reduces to encoding the query, since that comparison itself is a single matrix multiplication with negligible cost. The second limitation is the encoder: ViT-B/32 is the smallest Vision Transformer configuration of CLIP, chosen for speed. Replacing it with a larger configuration such as ViT-L/14 would be expected to raise both the similarity scores and the recall values of Table 9, at the price of a longer encoding time. The measurement protocol of Chapter 6 proves the baseline against which such an upgrade can be judged.

Three further directions extend the work rather than repair it. An approximate nearest-neighbour search method such as FAISS [15] would keep the search fast if the collection grew far beyond the 31,783 images used here for which the exhaustive comparison is still practical. Fine-tuning CLIP on Flickr30k would adapt the general-purpose embeddings to the everyday scenes of

this dataset, at the cost of the training that the present design deliberately avoids. Finally, repeating the Recall@K evaluation on the standard 1,000-image Flickr30k test split would make the results directly comparable with the published literature, which the nested subsets of this work do not allow.

7.3 Closing remarks

Taken as a whole, the thesis demonstrates that a working multimodal search system, one that retrieves images from textual descriptions alone, can be assembled from a pre-trained dual encoder, cosine similarity and the captions already present in the dataset, without training any model. The pre-trained shared embedding space carries enough semantic structure that ordering by cosine similarity returns plausible images for arbitrary text over more than thirty thousand candidates and a second stage built only on the captions of the dataset is enough to correct its characteristic failures. The system runs interactively on consumer hardware and its two stages, fast visual retrieval followed by more precise re-ranking, reproduce in miniature the architecture of modern large-scale retrieval systems.

This thesis benefited from the assistance of an artificial intelligence tool, Claude (Anthropic), during its drafting and revision stages. The tool was used to refine the language for clarity, precision and accurate translation of key content. All ideas, implementation work and experimental results are the author's own, the AI assistance served to improve the coherence, correctness and readability of the final text.

8. References

- [1] A. Dosovitskiy et al., “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [2] A. Vaswani et al., “Attention Is All You Need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [3] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 139, 2021, pp. 8748–8763.
- [4] F. Faghri, D. J. Fleet, J. R. Kiros, and S. Fidler, “VSE++: Improving Visual-Semantic Embeddings with Hard Negatives,” in *Proc. British Machine Vision Conference (BMVC)*, Newcastle, 2018.
- [5] K.-H. Lee, X. Chen, G. Hua, H. Hu, and X. He, “Stacked Cross Attention for Image-Text Matching,” in *Proc. European Conference on Computer Vision (ECCV)*, Lecture Notes in Computer Science, vol. 11208, 2018, pp. 201–216.
- [6] J. Lu, D. Batra, D. Parikh, and S. Lee, “ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 13–23.
- [7] Y.-C. Chen et al., “UNITER: UNiversal Image-TExt Representation Learning,” in *Proc. European Conference on Computer Vision (ECCV)*, Lecture Notes in Computer Science, vol. 12375, 2020, pp. 104–120.
- [8] X. Li et al., “Oscar: Object-Semantics Aligned Pre-training for Vision-Language Tasks,” in *Proc. European Conference on Computer Vision (ECCV)*, Lecture Notes in Computer Science, vol. 12375, 2020, pp. 121–137.
- [9] C. Jia et al., “Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 139, 2021, pp. 4904–4916.
- [10] J. Li, D. Li, C. Xiong, and S. Hoi, “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2022.
- [11] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.
- [12] C.-W. Xie, S. Sun, X. Xiong, Y. Zheng, D. Zhao, and J. Zhou, “RA-CLIP: Retrieval Augmented Contrastive Language-Image Pre-training,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19265–19274.
- [13] M. Cherti et al., “Reproducible Scaling Laws for Contrastive Language-Image Learning,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 2818–2829.
- [14] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2020, pp. 38–45.

- [15] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [16] P. Young, A. Lai, M. Hodosh, and J. Hockenmaier, “From Image Descriptions to Visual Denotations: New Similarity Metrics for Semantic Inference over Event Descriptions,” *Trans. Assoc. Comput. Linguistics (TACL)*, vol. 2, pp. 67–78, 2014.
- [17] X. Chen et al., “Microsoft COCO Captions: Data Collection and Evaluation Server,” *arXiv preprint arXiv:1504.00325*, 2015.
- [18] P. Sharma, N. Ding, S. Goodman, and R. Soricut, “Conceptual Captions: A Cleaned, Hypernymed, Image Alt-text Dataset for Automatic Image Captioning,” in *Proc. 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2556–2565.
- [19] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki, “LAION-400M: Open Dataset of CLIP-Filtered 400 Million Image-Text Pairs,” in *Proc. NeurIPS Workshop on Data-Centric AI*, 2021.
- [20] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [21] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.
- [22] C. R. Harris et al., “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, Sep. 2020.
- [23] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, May–June 2007.
- [24] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:1906.02569*, 2019.

9. Appendix

9.1 Dataset preparation notebook

```
import pandas as pd
import matplotlib.pyplot as plt
import os
from PIL import Image
import random

## Function to show 3 random images with their captions

def show_3_images(df, img_folder):
    imgs = random.sample(list(df['image_name'].unique()), 3)
    fig, axes = plt.subplots(3, 1, figsize=(8, 15))
    for ax, img_name in zip(axes, imgs):
        img = Image.open(os.path.join(img_folder, img_name))
        ax.imshow(img)
        ax.axis('off')
        captions = df[df['image_name'] == img_name]['comment'].values
        titlu = "\n".join(f"{i+1}. {c}" for i, c in enumerate(captions))
        ax.set_title(titlu, fontsize=8, loc='left')
    plt.tight_layout()
    plt.show()

## Script for showing specific dataset data, and making sure everything is in
order(no missing images, captions etc)

# additional information:
# there are no train/val/test splits, they need to be created.
# data type = int64
# total captions = 158915
# image dimensions are variable but observed to not exceed 500px width or height,
format is .jpg
# the dataset is split into: image name | id (0-4) | caption
# there was a mistake in the dataset at line 19999 where a "|" separator was
missing, but I fixed it manually
# in the dataset there was a copy of results.csv and one for images, but I
deleted them to be more organized

PATH_CSV = r"C:\licenta\dataset\results.csv"
PATH_IMGS = r"C:\licenta\dataset\flickr30k_images"

df = pd.read_csv(PATH_CSV, sep='|', on_bad_lines='skip', skipinitialspace=True)

# Statistics

print(f"We have in total {df['image_name'].nunique()} unique images.")
```

```

df['word_count'] = df['comment'].astype(str).apply(lambda x: len(x.split()))

print(f"Average length of a description: {df['word_count'].mean():.2f} words.")
print(f"The longest description has {df['word_count'].max()} words.")

# Distribution of description lengths

plt.figure(figsize=(10, 5))
plt.hist(df['word_count'], bins=30, color='#3498db', edgecolor='black',
alpha=0.7)
plt.title("Distribution of Description Length", fontsize=14)
plt.xlabel("Number of Words", fontsize=12)
plt.ylabel("Number of Descriptions", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

# We select a random image
random_img_name = random.choice(df['image_name'].unique())
full_path = os.path.join(PATH_IMGS, random_img_name)

# we show image and captions for the random image, to get a sense of the data
if os.path.exists(full_path):
    img = Image.open(full_path)
    plt.figure(figsize=(7, 7))
    plt.imshow(img)
    plt.axis('off')
    plt.title(f"Exemplu: {random_img_name}")
    plt.show()

    print(f"\n Captions pentru imaginea '{random_img_name}':")
    captions = df[df['image_name'] == random_img_name]['comment'].values
    for i, cap in enumerate(captions, 1):
        print(f"    {i}. {cap}")
else:
    print(f"\nNu exista {random_img_name} la calea specificată.")

# Verification of dataset data

print("\nVerification of dataset data:")

# Verification of dataset data
print("Missing values:", df.isnull().sum().sum())          # 0 -> no empty
captions

csv_imgs      = set(df['image_name'].unique())
folder_imgs   = set(os.listdir(PATH_IMGS))

```

```

missing_files = csv_imgs - folder_imgs          # images referenced
but absent
print("Missing files:", len(missing_files))      # 0 -> all images
present

show_3_images(df, PATH_IMGS)

```

9.2 Baseline implementation notebook

```

## Setup Cells, imports, loading the model, loading all the files
import os # for file/folder operations (paths, creating directories, etc.)
import csv # for reading/writing CSV files (used for handling the results of the
evaluation)
import torch # Pytorch library for deep learning used for all tensor operations
and model handling
import pandas as pd # for data manipulation and analysis (used to manipulate the
dataset CSV)
import matplotlib.pyplot as plt # plotting the image grid
from PIL import Image # to open image files as PIL(pillow) objects before feeding
to CLIP
from transformers import CLIPProcessor, CLIPModel # the actual CLIP model and
its preprocessor from HuggingFace transformers library
import torch.nn.functional as F # we use this specifically for F.normalize() - L2
normalization of embeddings, for the cosine similarity calculation
import gradio as gr # used for the interactive GUI at the end

device = "cuda" if torch.cuda.is_available() else "cpu" # use GPU if available
for faster processing power
print(f"Device: {device}")

IMAGE_DIR = r"C:\licenta\dataset\flickr30k_images" # folder where images are
stored
CSV_PATH = r"C:\licenta\dataset\results.csv" # path to the CSV files which
contains the image filenames and corresponding captions
OUTPUT_DIR = r"C:\licenta\rezultate_evaluate" # folder where we save the
evaluation results (images with captions and similarity scores)
os.makedirs(OUTPUT_DIR, exist_ok=True) # creates the output directory if it
doesn't exist already (exist_ok=True prevents errors(crashes if it doesnt exist))

TOP_K = 5 # the number of top results we return for each query
BATCH_SIZE = 32 # the number of images we process at once when computing
embeddings (helps with memory management and speed), we can increase it based on
the VRAM available

# We load a pretrained CLIP model
model_id = "openai/clip-vit-base-patch32" # the specific model we use VIT-B/32
decent

```

```

model = CLIPModel.from_pretrained(model_id).to(device) # load the model and move
it to the device
processor = CLIPProcessor.from_pretrained(model_id) # the processor handles
resizing and normalizing images + tokenizing text
model.eval() # sets model into evaluation mode
print(f"Model loaded: {model_id}")

# Reading the dataset CSV file
df = pd.read_csv(CSV_PATH, sep="|") # using panda to read the CSV file, sep="|"
because the columns are separated by pipe character in our CSV
df.columns = df.columns.str.strip() # strip() removes any leading or trailing
whitespace from the column names, ensuring we can access them without issues
(e.g., df['image_name'] instead of df[' image_name '])

all_images = df['image_name'].unique() # we get the unique image names from the
dataset CSV
print(f"Total images : {len(all_images)}") # the length of the dataset, should be
31,783 images

# 8 queries we use for test retrieval across all dataset subset sizes (1k, 5k,
10k, 20k, 30k)
# they are varied in term of content and complexity in order to test the model
performance across different types of queries

test_queries = [
    "a dog playing in the park while a woman watches nearby",
    "people walking on a city street with umbrellas while laughing",
    "a child holding a red balloon in a field of flowers",
    "sunset over a mountain range with a lake in the foreground",
    "a group of friends having a picnic in the park on a sunny day",
    "a woman sitting at a cafe table with a laptop and a cup of coffee",
    "cars on a busy street with tall buildings in the background",
    "group of people playing soccer on a grassy field with trees around it",
]

## Funtions for the thesis project

# Loads all images from IMAGE_DIR, converts them to RGB, runs them through CLIP's
image encoder
# in batches, L2 normalizes the output vectors and returns a tensor of shape [N,
512] + the valid paths

def encode_images(image_names):

    images = [] # holds the loaded PIL image objects

```

```

paths = [] # holds the corresponding file paths for the loaded images (used
for displaying results and saving to CSV)
for img_name in image_names:
    img_path = os.path.join(IMAGE_DIR, img_name.strip()) # builds the full
path, strip() used in order to remove any whitespace
    if not os.path.exists(img_path):
        continue # if the image file doesnt exist we skip it and move on
    try:
        images.append(Image.open(img_path).convert("RGB")) #opens the images
and forces to be RGB format (CLIP expects RGB) we convert in order to avoid
errors
        paths.append(img_path)
    except Exception as e:
        print(f" Error at {img_name}: {e}") # if an image fails to load we
print the error and move on

total = len(images)
print(f" {total} images loaded, processing in batches of {BATCH_SIZE}")

# processing in batches for speed
embeddings = []
with torch.no_grad(): #no_grad() tells PyTorch to not build a computation
graph
    for start in range(0, total, BATCH_SIZE): # iterate in steps of
BATCH_SIZE
        batch = images[start:start + BATCH_SIZE] # we grab the current batch
of pillow images
        inputs = processor(images=batch, return_tensors="pt",
padding=True).to(device) # we resize, normalize and convert images into tensors,
send them to device after
        output =
model.get_image_features(pixel_values=inputs['pixel_values']) # run through CLIP
image encoder only
        features = output.pooler_output if hasattr(output, 'pooler_output')
else output # this will give always else for our case or if we use a different
CLIP variant that has pooler_output, we will use that instead (some models have
it, some dont)
        features = F.normalize(features, p=2, dim=-1) # L2 normalize the
features to unit length, we use this for cosine similarity
        embeddings.append(features.cpu()) # move back to CPU to free up VRAM

        done = min(start + BATCH_SIZE, total)
        if done % 500 < BATCH_SIZE or done == total:
            print(f" {done}/{total} processed") # this is for logging every
around 500 images

embeddings = torch.cat(embeddings, dim=0) # we concatenate all batch tensors
into one big tensor of matrix shape [num_images(N), embedding_dim(512 for ViT-
B/32)]
print(f" Gata: {embeddings.shape[0]} embeddings ({embeddings.shape[1]}D)")

```

```

    return embeddings, paths ## return the embedding matrix and the list of paths

# Takes a text query, encodes it with CLIP's text encoder, then computes cosine
similarity
# against all image embeddings via dot product and returns the TOP_K most similar
images
def search(query, image_embeddings, valid_paths, top_k=TOP_K):
    inputs = processor(text=[query], return_tensors="pt",
padding=True).to(device) # this is the tokenization of the text query

    with torch.no_grad():
        output = model.get_text_features(**inputs) # we run only the text encoder
of CLIP to get the text embedding for the query
        text_features = output.pooler_output if hasattr(output, 'pooler_output')
else output
        text_features = F.normalize(text_features, p=2, dim=-1).cpu() # same L2
normalization used for images, used for cosine similarity, move back to CPU for
the similarity calculation

        similarities = torch.matmul(text_features, image_embeddings.T).squeeze(0)
        # matrix multiply text vector [1, 512] with the transpose of image embeddings
[512, N] to get a similarity score for each image, resulting in a vector of shape
[N]
        # .squeeze(0) removes the extra batch dimension so we get a flat 1D tensor
        top_indices = similarities.topk(top_k).indices # get indices of the TOP_K
highest scores

    results = []
    for i, idx in enumerate(top_indices):
        results.append({
            "rank": i + 1, # rank starts at 1 for better readability
            "image_name": os.path.basename(valid_paths[idx]), # we extract just
the filename from the full path for display and saving to CSV
            "image_path": valid_paths[idx], # full path for loading the image
later
            "similarity_score": round(similarities[idx].item(), 6), # we use
.item() in order to convert the tensor scalar to a Python float
        })
    return results

# Saves the results dict { query -> [top-K results] } to a CSV file in OUTPUT_DIR
# filename is automatically named after the subset size e.g.
rezultate_500_imagini.csv
def save_csv(all_results, subset_size, top_k):
    path = os.path.join(OUTPUT_DIR, f"rezultate_{subset_size}_imagini.csv") #
dynamic filename based on subset size

```

```

    with open(path, 'w', newline='', encoding='utf-8') as f: # newline='' is
required by Python's csv module on Windows
        writer = csv.writer(f)
        writer.writerow(['subset_size', 'query', 'rank', 'image_name',
'image_path', 'similarity_score', 'top_k']) # header row
        for query, results in all_results.items(): # all_results is a dict: {
query_string -> list of result dicts }
            for r in results:
                writer.writerow([subset_size, query, r['rank'], r['image_name'],
r['image_path'], r['similarity_score'], top_k])
            print(f"Saved {path}")

```

Encodes all images in the given subset and runs search() for every query in the list

Returns a dict { query -> [top-K result dicts] } – the main format used throughout the notebook

```
def evaluate_subset(image_names, queries):
```

```
    embeddings, paths = encode_images(image_names) # encode all images in this
subset first to get the embeddings and valid paths
```

```
    if len(paths) == 0:
        print("Error no images found.")
        return {} # avoiding tensor errors
```

```
    results = {}
    for q in queries:
        results[q] = search(q, embeddings, paths, top_k) # run search for each
query against the image embeddings and store them into a dict: { query_string ->
list of result dicts }
        top = results[q][0]
        print(f"  '{q}' -> {top['image_name']} (score:
{top['similarity_score']:.4f})") # print the top result for each query for quick
feedback during evaluation
```

```
    return results
```

Displays the top-K retrieved images for each query as a horizontal image grid
Each image shows its rank, similarity score and filename as a title

```
def plot_results(results, subset_label, top_k):
```

```
    for q in results: # iterate for one figure per query
```

```

fig, axes = plt.subplots(1, top_k, figsize=(4 * top_k, 4)) # one row,
TOP_K columns, each subplot 4x4 inches

for i, r in enumerate(results[q]):
    img = Image.open(r["image_path"])
    axes[i].imshow(img)
    axes[i].set_title(
        f"#{r['rank']} score:
{r['similarity_score']:.3f}\n{r['image_name']}",
        fontsize=14
    )
    axes[i].axis("off") # we turn off the axes for a cleaner look since
we only care about the images and their titles

```

```

fig.suptitle(f"[{subset_label}] {q}", fontsize=18)
plt.tight_layout()
safe_q = q.replace(" ", "_").replace("/", "-")[:40]
plt.savefig(os.path.join(OUTPUT_DIR,
f"rezultate_{subset_label}_{safe_q}.png"),
            dpi=120, bbox_inches="tight")
plt.show()

```

Computes overlap and average similarity scores between two subsets for each query

Saves a comparison CSV to OUTPUT_DIR with per-query stats

```

def compare_results(results_A, results_B, label_A, label_B, top_k, output_dir):
    print(f"Comparison {label_A} vs {label_B}\n")
    comparison_rows = []

```

```

    for q in results_A:
        # imaginile returnate de fiecare subset
        imgs_A = set(r["image_name"] for r in results_A[q]) # set of image names
from subset A
        imgs_B = set(r["image_name"] for r in results_B[q]) # set of image names
from subset B
        overlap = imgs_A & imgs_B # this is the intersection of iamges that
appear in both subsets for this query

```

```

        avg_A = sum(r["similarity_score"] for r in results_A[q]) / top_k # mean
similarity score for subset A

```

```

        avg_B = sum(r["similarity_score"] for r in results_B[q]) / top_k # mean
similarity score for subset B

```

```

        # print overlap count and average scores for quick comparison
        print(f"'{q}'")
        print(f"  Overlap : {len(overlap)}/{top_k} Common images in top-{top_k}
results")

```

```

        print(f" Average score : {avg_A:.4f} ({label_A}) -> {avg_B:.4f}
({label_B})")
        if overlap:
            print(f" Common : {' , '.join(sorted(overlap))}")
        print()

    comparison_rows.append({ # collect data for CSV export
        "query"           : q,
        "subset_A"        : label_A,
        "subset_B"        : label_B,
        "overlap_count"   : len(overlap),
        "top_k"           : top_k,
        "avg_score_A"     : round(avg_A, 6),
        "avg_score_B"     : round(avg_B, 6),
        "overlap_images"  : "; ".join(sorted(overlap)),
    })

    # save the comparison as a CSV(maybe usefull for results tables)
    comp_path = os.path.join(output_dir, f"comparatie_{label_A}-{label_B}.csv")
    with open(comp_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=comparison_rows[0].keys()) #
DictWriter uses dict keys as column names automatically
        writer.writeheader()
        writer.writerows(comparison_rows)

    print(f"Comparatie salvata: {comp_path}")

# Displays two rows of images side by side for a single query, top row is subset
A, bottom is subset B
# Images that appear in both subsets get a green border and a checkmark, grey
border otherwise
# Saves the figure as a PNG to OUTPUT_DIR
def plot_comparison(query, results_A, results_B, label_A, label_B, top_k,
output_dir, compare_scores=True):

    res_A = results_A[query]
    res_B = results_B[query]

    common = set(r["image_name"] for r in res_A) & set(r["image_name"] for r in
res_B) # images that show up in both subsets (intersection for common images)

    avg_A = sum(r["similarity_score"] for r in res_A) / top_k
    avg_B = sum(r["similarity_score"] for r in res_B) / top_k

    fig, axes = plt.subplots(2, top_k, figsize=(4 * top_k, 9)) # 2 rows (one per
subset), TOP_K columns
    fig.patch.set_facecolor("#f5f5f5") # light grey background for the whole
figure

```

```

for row_idx, (results, label, avg) in enumerate([
    (res_A, label_A, avg_A),
    (res_B, label_B, avg_B),
]):
    for col_idx, r in enumerate(results):
        ax = axes[row_idx][col_idx]

        try:
            ax.imshow(Image.open(r["image_path"]))
        except Exception:
            ax.text(0.5, 0.5, "N/A", ha="center", va="center",
transform=ax.transAxes) # if image can't load, show "N/A" placeholder

        is_common = r["image_name"] in common # check if this image appears
in both subsets
        for spine in ax.spines.values():
            spine.set_edgecolor("#2ecc71" if is_common else "#cccccc") #
green border for common images, grey otherwise
            spine.set_linewidth(4 if is_common else 1)

        title = f"#[{r['rank']}] score:
{r['similarity_score']:.3f}\n[{r['image_name']}]"
        ax.set_title(
            title + (" *" if is_common else ""), # add a checkmark to the
title if it's a common result
            fontsize=14,
            color="#2ecc71" if is_common else "black"
        )
        ax.axis("off")

        # label for the row with the subset name and average score, rotated to
read horizontally
        row_label = f"{label} | avg score: {avg:.4f}" if compare_scores else
label
        axes[row_idx][0].text(
            0.0, 1.32, row_label,
            transform=axes[row_idx][0].transAxes,
            fontsize=15, fontweight="bold", ha="left", va="bottom"
        )

    suptitle = (
        f'Query: "{query}"\n'
        f'Overlap: {len(common)}/{top_k} ({len(common) / top_k * 100:.0f}%)'
    )
    if compare_scores:
        # subset vs subset: both rows are (query <-> image) scores, so the diff
is meaningful
        suptitle += f' | difference in average score {avg_B - avg_A:+.4f}' #
the + forces the sign (+0.0123 / -0.0045)

```

```

else:
    # baseline (query <-> image) vs reranked (query <-> caption): different
    scales,
    # so we deliberately do NOT compare the scores numerically
    suptitle += ' | scores not directly comparable (image-sim vs caption-
sim)'

fig.suptitle(suptitle, fontsize=16, fontweight="bold", y=1.01)

plt.tight_layout(h_pad=4) # was plt.tight_layout() – h_pad adds space
between rows

# salvam figura
safe_q = query.replace(" ", "_").replace("/", "-")[:40] # make the query safe
to use as a filename, replace spaces/slashes and cap at 40 chars
plt.savefig(
    os.path.join(output_dir, f"comparatie_{label_A}-{label_B}_{safe_q}.png"),
    dpi=120, bbox_inches="tight" # bbox_inches="tight" prevents the suptitle
from being clipped
)
plt.show()

# Cross-references retrieved images with the original Flickr30k captions from the
CSV
# Lets you manually verify if the images CLIP returned actually match the query
semantically
def verify_captions(results, subset_label, top_n=5):
    print(f>Description of the returned images ({subset_label}):\n")
    # function used to check if the images CLIP retrieved actually match the
    query by looking at the captions associated with those images in the dataset CSV

    for q in results:
        print(f>Query: '{q}')
        for r in results[q][:top_n]: # only check the top 5 results by default
            captions = df[df["image_name"].str.strip() ==
r["image_name"]]["comment"].values # we look up the captions for this image in
the original dataset CSV, we use str.strip() to ensure we match the filename
correctly without whitespace issues
            print(f"    #{r['rank']} {r['image_name']} (score:
{r['similarity_score']})")
            for cap in captions[:top_n]:
                print(f"        -> {str(cap).strip()}")
            print()

```

```

# Computes Recall@K (R@1, R@5, R@10) for text -> image retrieval, the standard
metric from the literature
# We use the real Flickr30k captions as queries: each caption already knows which
image it describes,
# so that image is the ground truth (the correct answer) we check for in the top-
K results
def evaluate_recall(image_names, k_values=(1, 5, 10), text_batch=256):

    embeddings, paths = encode_images(image_names) # encode the image subset
    once, reuse for every caption query
    name_to_idx = {os.path.basename(p): i for i, p in enumerate(paths)} # map
    each filename to its row in the embedding matrix
    present = set(name_to_idx) # the set of images we actually managed to load
    (some might be missing)

    # we build pairs of (caption, index of its correct image) only for images
    that exist in this subset
    captions, gt_idx = [], [] # captions = the query texts, gt_idx = the ground-
    truth image index for each one
    sub = df[df["image_name"].str.strip().isin(present)] # keep only the rows
    whose image is in our subset
    for name, cap in zip(sub["image_name"].str.strip(), sub["comment"]):
        cap = str(cap).strip() # clean up the caption, str() in case it's not
        already a string
        if cap: # skip empty captions so we don't query with nothing
            captions.append(cap)
            gt_idx.append(name_to_idx[name]) # remember which image this caption
            belongs to

    max_k = max(k_values) # we only need to retrieve up to the largest K (10),
    then check 1/5/10 from it
    hits = {k: 0 for k in k_values} # counts how many queries had their correct
    image inside the top-K

    with torch.no_grad(): # no gradients needed, we are only doing inference
        for start in range(0, len(captions), text_batch): # process captions in
            batches for speed (like encode_images)
            batch = captions[start:start + text_batch] # the current batch of
            caption queries
            inputs = processor(text=batch, return_tensors="pt", padding=True,
            truncation=True).to(device) # tokenize + send to device
            output = model.get_text_features(**inputs) # run only the text
            encoder of CLIP
            tf = output.pooler_output if hasattr(output, 'pooler_output') else
            output # same feature extraction as search()/encode_images
            tf = F.normalize(tf, p=2, dim=-1).cpu() # L2 normalize for cosine
            similarity, move back to CPU
            sims = torch.matmul(tf, embeddings.T) # [B, N] similarity of every
            caption in the batch against all images

```

```

        topk = sims.topk(max_k, dim=1).indices # [B, max_k] the top image
indices for each caption
        for row, gi in enumerate(gt_idx[start:start + text_batch]): # for
each caption in the batch
            ranked = topk[row].tolist() # its ranked list of top image
indices
            for k in k_values:
                if gi in ranked[:k]: # if the correct image is inside the
top-K, it counts as a hit
                    hits[k] += 1

    n = len(captions) # total number of caption queries we evaluated
    recall = {k: hits[k] / n for k in k_values} # recall = hits / total, for
each K
    print(f"Recall on {len(image_names)} images, {n} caption-queries:")
    for k in k_values:
        print(f" R@{k}: {recall[k]*100:.2f}% ({hits[k]}/{n})") # print as
percentage + raw hit count
    return recall # return the dict { K -> recall } so we can reuse it (e.g.
tables, comparisons)

def load_results(csv_name):
    path = os.path.join(OUTPUT_DIR, csv_name)
    df = pd.read_csv(path, header=None, skiprows=1,
                      names=["subset_size", "query", "rank", "image_name",
                             "image_path", "similarity_score", "top_k"])
    return {q: grp.sort_values("rank").to_dict("records")
            for q, grp in df.groupby("query", sort=False)}

## Evaluation for 500 images
results_500 = evaluate_subset(all_images[:500], test_queries)
plot_results(results_500, subset_label="500 img", top_k=TOP_K)

## Evaluation for 5000 images

print("Evaluate pe 5000 imagini")
results_5000 = evaluate_subset(all_images[:5000], test_queries)
plot_results(results_5000, subset_label="5000 img", top_k=TOP_K)

## Evaluation for 10000 images
print("Evaluate pe 10000 imagini")
results_10000 = evaluate_subset(all_images[:10000], test_queries)
plot_results(results_10000, subset_label="10000 img", top_k=TOP_K)

## Evaluation for 20000 images
print("Evaluate pe 20000 imagini ")
results_20000 = evaluate_subset(all_images[:20000], test_queries)
plot_results(results_20000, subset_label="20000 img", top_k=TOP_K)

```

```

## Evaluation for all images
print("Evaluare pe toate cele 31783 imagini")
results_all = evaluate_subset(all_images, test_queries)
plot_results(results_all, subset_label="toate img", top_k=TOP_K)

## Saving to csv
save_csv(results_500, 500, TOP_K)
save_csv(results_5000, 5000, TOP_K)
save_csv(results_10000, 10000, TOP_K)
save_csv(results_20000, 20000, TOP_K)
save_csv(results_all, "toate imaginile", TOP_K)

## Comparing 500-5000 images
compare_results(results_500, results_5000, label_A="500 img", label_B="5000 img",
top_k=TOP_K, output_dir=OUTPUT_DIR)

for q in test_queries:
    plot_comparison(q, results_500, results_5000, label_A="500 img",
label_B="5000 img", top_k=TOP_K, output_dir=OUTPUT_DIR)

## Comparison 5000 vs 10.000 images
for q in test_queries:
    plot_comparison(q, results_5000, results_10000, label_A="5000 img",
label_B="10000 img", top_k=TOP_K, output_dir=OUTPUT_DIR)

## Comparison 10.000 vs 20.000 images
for q in test_queries:
    plot_comparison(q, results_10000, results_20000, label_A="10000 img",
label_B="20000 img", top_k=TOP_K, output_dir=OUTPUT_DIR)

## Comparison 20.000 vs all images
for q in test_queries:
    plot_comparison(q, results_20000, results_all, label_A="20000 img",
label_B="toate imaginile", top_k=TOP_K, output_dir=OUTPUT_DIR)

## Verification with real captions

results_all = load_results("rezultate_toate imaginile_imagini.csv")

selected = {q: results_all[q] for q in [
    "a dog playing in the park while a woman watches nearby",
    "a woman sitting at a cafe table with a laptop and a cup of coffee",
]}
verify_captions(selected, "all images")

```

```

def rerank_with_captions(query, image_embeddings, valid_paths, df):

    # standard search from above
    initial_results = search(query, image_embeddings, valid_paths)

    # encode the query text into a vector we do this once and reuse it for all 5
    images
    query_input = processor(
        text=[query],
        return_tensors="pt", # return tensors pytorch
        padding=True,
        truncation=True # truncate if the query is longer than the models
max token limit 77 tokens for CLIP
    ).to(device)

    with torch.no_grad(): # no gradient computation
        output = model.get_text_features(**query_input)
        query_vec = output.pooler_output if hasattr(output, 'pooler_output') else
output # handle both tensor and object return types
        query_vec = F.normalize(query_vec, p=2, dim=-1).cpu() # L2 normalize so
dot product, cosine similarity, move to CPU

    reranked = []
    for r in initial_results: # iterate over each of the 5 images returned by
CLIP
        image_name = r["image_name"]

        # fetch all real human written captions for this image from the dataset
        captions = df[df["image_name"].str.strip() ==
image_name]["comment"].values
        captions = [str(c).strip() for c in captions]

        if len(captions) == 0:
            # if no captions are found, fall back to the original CLIP score so
the image still appears in results
            reranked.append(**r, "similarity_score": r["similarity_score"])
            continue

        # all captions for this image together in one batch
        caption_inputs = processor(
            text=captions,
            return_tensors="pt",
            padding=True,
            truncation=True
        ).to(device)

        with torch.no_grad():

```

```

        output = model.get_text_features(**caption_inputs)
        caption_vecs = output.pooler_output if hasattr(output,
'pooler_output') else output
        caption_vecs = F.normalize(caption_vecs, p=2, dim=-1).cpu() # [5,
512] one vector per caption

        # compare the query vector against each of the 5 caption vectors
        # [1, 512] x [512, 5] = [5] one similarity score per caption
        scores = torch.matmul(query_vec, caption_vecs.T).squeeze(0)

        # keep only the best matching caption score for this image

        max_score = scores.max().item()

        reranked.append({
            "rank": r["rank"],          # placeholder rank, will be reassigned
after sorting
            "image_name": image_name,
            "image_path": r["image_path"],
            "similarity_score": round(max_score, 6) # replace CLIP visual score
with the text-based rerank score
        })

        # sort the 5 images by their new caption based score in descending order
        reranked.sort(key=lambda x: x["similarity_score"], reverse=True)

        # reassign ranks after sorting rank 1 = best caption match to the query
        for i, r in enumerate(reranked):
            r["rank"] = i + 1

        # return both the original CLIP results and the reranked results for
comparison
        return initial_results, reranked

def show_rerank_changes(query, baseline, reranked):
    """
    # Compares the ranking BEFORE and AFTER caption-reranking for one query.
    # Reranking only reorders the same top-K images (it doesn't add new ones),
    # so what matters is how each image's POSITION changed.
    # Scores are shown separately and are NOT comparable:
    # - score img      = baseline (query <-> image)
    # - score caption = reranked (query <-> caption)
    """
    base_rank = {r["image_name"]: r["rank"] for r in baseline} #
rank of each image before rerank
    base_score = {r["image_name"]: r["similarity_score"] for r in baseline} #
baseline (query <-> image) score

```

```

print(f'\nQuery: "{query}"')
print(f'{"image":35} {"rank_base":>9} {"rank_rerank":>11} {"movement":>9} '
      f'{"score_img":>9} {"score_caption":>13}')
print("-" * 95)

for r in reranked:
    name = r["image_name"]
    old = base_rank[name] # position before rerank (query <-> image)
    new = r["rank"] # position after rerank (query <-> caption)
    delta = old - new # > 0 means the image moved UP

    if delta > 0: move = f"up {delta}"
    elif delta < 0: move = f"down {abs(delta)}"
    else: move = "same"

    print(f'{"name":35} {"old":>9} {"new":>11} {"move":>9} '
          f'{"base_score[name]":>9.4f} {"similarity_score":>13.4f}')

# Re-ranking on several queries (at least 3) so the results are more
representative
rerank_queries = [
    "a dog playing in the park while a woman watches nearby",
    "a child holding a red balloon in a field of flowers",
    "a woman sitting at a cafe table with a laptop and a cup of coffee",
]

print("Reranking with captions on 5000 imagini")
embeddings_5000, paths_5000 = encode_images(all_images[:5000]) # encode once,
reuse for every query

for QUERY in rerank_queries:
    print("\n" + "=" * 80)
    print(f"QUERY: {QUERY}")
    print("=" * 80)

    baseline, reranked = rerank_with_captions(QUERY, embeddings_5000, paths_5000,
df)

    # comparatie text
    baseline_names = set(r["image_name"] for r in baseline)
    reranked_names = set(r["image_name"] for r in reranked)
    overlap = baseline_names & reranked_names

    print(f"\nBaseline top {TOP_K}:")
    for r in baseline:
        print(f"  #{r['rank']} {r['image_name']} (score:
{r['similarity_score']:.4f})")

```

```

print(f"\nReranked top {TOP_K}:")
for r in reranked:
    print(f"  #{r['rank']} {r['image_name']} (score: {r['similarity_score']:.4f})")

print(f"\nOverlap: {len(overlap)}/{TOP_K} common images in top {TOP_K} results")
if overlap:
    print(f"Common images: {' '.join(sorted(overlap))}")

# before -> after rank comparison (shows how the ranking changed)
show_rerank_changes(QUERY, baseline, reranked)

# visual comparison
results_baseline = {QUERY: baseline}
results_reranked = {QUERY: reranked}

plot_comparison(
    QUERY,
    results_baseline,
    results_reranked,
    label_A="CLIP baseline",
    label_B="caption reranked",
    top_k=TOP_K,
    output_dir=OUTPUT_DIR,
    compare_scores=False, # scores aren't comparable -> don't show the
score diff
)

## R@K evaluation

recall_1000 = evaluate_recall(all_images[:1000])
recall_5000 = evaluate_recall(all_images[:5000])
recall_20000 = evaluate_recall(all_images[:20000])

## GUI DEMO

def gradio_search(query, subset_size, top_k):
    image_names = all_images[:int(subset_size)] # slice the dataset to the
selected size
    embeddings, paths = encode_images(image_names) # encode on the fly (slow for
large subsets but fine for a demo)

    results = search(query, embeddings, paths, top_k=int(top_k)) # run search and
take top_k results

```

```

images = []
captions = []
for r in results:
    try:
        img = Image.open(r["image_path"]).convert("RGB")
        images.append(img)
        captions.append(f"#{r['rank']} score: {r['similarity_score']:.3f} |
{r['image_name']}")
    except Exception:
        pass # silently skip any image that fails to load

return list(zip(images, captions)) # Gradio Gallery expects a list of (image,
caption) tuples

```

```

demo = gr.Interface(
    fn=gradio_search, # the function to call when the user submits
    inputs=[
        gr.Textbox(label="Query text"),
        gr.Slider(100, 31783, value=5000, step=100, label="Subset images"), #
slider to select how many images from the dataset to include in the search (from
100 to all 31783, default 5000)
        gr.Slider(1, 5, value=5, step=1, label="Top-K results"),
    ],
    outputs=gr.Gallery(label="Results"), # displays the results as a scrollable
image gallery
    title="CLIP Image Search",
    description="Search images in Flickr30k using a text query."
)

```

```
demo.launch()
```

```
## Loading and comparing results from CSV files (for thesis report)
```

```

results_500   = load_results("rezultate_500_imagini.csv")
results_5000  = load_results("rezultate_5000_imagini.csv")
results_10000 = load_results("rezultate_10000_imagini.csv")
results_20000 = load_results("rezultate_20000_imagini.csv")
results_all   = load_results("rezultate_toate_imaginile_imagini.csv")

```

```

pairs = [(results_500, results_5000, "500 img", "5000 img"),
         (results_5000, results_10000, "5000 img", "10000 img"),
         (results_10000, results_20000, "10000 img", "20000 img"),
         (results_20000, results_all, "20000 img", "all images")]

```

```

for res_A, res_B, lab_A, lab_B in pairs:
    compare_results(res_A, res_B, lab_A, lab_B, TOP_K, OUTPUT_DIR)
    for q in res_A:
        plot_comparison(q, res_A, res_B, lab_A, lab_B, TOP_K, OUTPUT

```



Dragos-Daniel Degerat

Date of birth: 28/03/2003 | **Place of birth:** Targu Mures, Romania | **Nationality:** Romanian |

Gender: Male | **Phone number:** (+40) 0770374534 (Mobile) | **Email address:**

degeratdragos@gmail.com | **LinkedIn:** [Degerat Draogs-Daniel](#) |

Address: Strada Observatorului 34, Camin 5, Cluj-Napoca, Romania (Work)

EDUCATION AND TRAINING

03/10/2022 – CURRENT Cluj-Napoca, Romania

UNDERGRADUATE STUDENT UTCN, Faculty of Electronics, Telecommunications and Information Technology

Systems of communications and routing
Telephony
Computer Networks
Computer Programming and Programming Languages
Passive Components and Circuits
Electronic Devices
Computer-Aided Graphics
Digital Integrated Circuits
Software Engineering
Analog Integrated Circuit Systems
Web Technologies and Databases
Microprocessor Architecture
Information Transmission Theory

Address Strada George Barițiu 26-28, Cluj-Napoca 400027, Cluj-Napoca, Romania | **Website** <https://www.utcluj.ro/> |

Field of study Telecommunication Technologies and Systems

Târgu Mureș, Romania

HIGHSCHOOL GRADUATE National College "Alexandru Papiu Ilarian"

Address Strada, Piața Bernády György 12, 540072, Târgu Mureș, Romania | **Website** <http://www.papiu.ro/>

VOLUNTEERING

14/03/2024 – CURRENT Cluj-Napoca

Volunteer BEST Cluj-Napoca

Human Resources Positions

Feb. 2025 – Aug. 2025 (7 months) – Education

I contributed to building and supporting the organizing team for an international event dedicated to STEM education. I handled recruitment, team motivation, internal communication, and ensuring a welcoming and positive environment. This experience strengthened my teamwork, leadership, and people-oriented skills.

Fundraising Responsible

Oct. 2024 – Dec. 2024 (3 months) – Education

I was responsible for communicating with companies to identify mutually beneficial partnership opportunities. I contributed to securing sponsorships and promotional materials (vouchers, products, etc.).

Human Resources Positions

Jul. 2024 – Aug. 2024 (2 months) – Arts & Culture

As part of a summer school dedicated to international students interested in architecture, I organized cultural activities showcasing Romanian values and traditions, as well as themed social events that encouraged integration and group cohesion.

● **PRACTICE**

14/07/2025 – 02/09/2025
Intern Energobit

I was involved in activities designed to provide a broad perspective on the company's main areas of operation. I participated in technical presentations and visited departments such as substation design, protection and relay systems, SCADA, as well as renewable energy solutions. I also toured the production area for energy equipment. The program included mentoring sessions and training focused on developing interpersonal and teamwork skills. This experience gave me a valuable combination of technical and practical knowledge about how the energy industry operates.

● **CERTIFICATIONS**

Emerson Test & Measurement University Relations Team,, 19/11/2025
Level Up Your Communication: Assertiveness Matters

I deepened my skills in clear communication, constructive handling of difficult situations, and adapting messages to different audiences. The knowledge I gained helped me improve my teamwork and my ability to convey ideas in a professional, results oriented way.

Mode of learning: Online

Link <https://europa.eu/europass/eportfolio/screen/share/1b191d21-6887-44f9-8799-a28122bd84cb?lang=ro>

● **HONOURS AND AWARDS**

18/02/2024
Second place Invent for the Planet 2024 – Technical University Cluj-Napoca

An event organized simultaneously across more than 30 university campuses worldwide, focused on solving major contemporary issues. The competition involves designing an innovative prototype to address these problems within 48 hours. Some of the skills I obtained during this competition were teamwork, working under extreme stress and under a limited time

● **LANGUAGE SKILLS**

Mother tongue(s): **ROMANIAN**

Other language(s):

	UNDERSTANDING		SPEAKING		WRITING
	Listening	Reading	Spoken production	Spoken interaction	
ENGLISH	C1	C1	B2	C1	B2

Levels: A1 and A2: Basic user; B1 and B2: Independent user; C1 and C2: Proficient user

● **SKILLS**

C++ | Python (computer programming) | Linux | Social Media | Web: HTML5, CSS3, JavaScript, PHP | Java (computer programming) | Microsoft Office | Microsoft PowerPoint | Microsoft Word | Microsoft Excel | Google Drive | Internet Explorer Edge Mozilla Google Chrome | Microsoft office word | MATLAB

● **DRIVING LICENCE**

Driving Licence: B | 28/10/2021 – 27/10/2031

